Chapter 25
Symbolic Trajectory Evaluation

Tom Melham

Abstract Symbolic trajectory evaluation is an industrial-strength formal hardware verification method, based on symbolic simulation, which has been highly successful in data-path verification, especially for microprocessor execution units. It is a 'model checking' method in the basic sense that properties, expressed in a simple temporal logic, are verified by (symbolic) exploration of formal models of sequential circuits. Its defining characteristic is that it operates by symbolic simulation over *abstractions* of sets of states that only partially delineate the circuit states in the set. These abstract state sets are ordered in a lattice by information content, based on a three-valued domain for values on circuit nodes (*true*, *false*, and *don't know*). The algorithm operates over families of these abstractions encoded by Boolean formulas, providing a flexible, specification-driven, mechanism for partitioned data abstraction. We provide a basic introduction to symbolic trajectory evaluation and its extensions, and some details of how it is deployed in industrial practice. The aim is to get across the essence and value of the method in clear and accessible terms.

25.1 Introduction

Symbolic Trajectory Evaluation (STE) is a model checking method based on symbolic simulation over a lattice of abstract state sets [64]. STE's combination of abstraction and algorithmic efficiency is especially suited to verification of large datapaths and memories, and has been demonstrated on many hard industrial verification problems at Intel Corporation [43, 53, 59]. Two notable successes are the verification, using Intel's Forte system [65], of the entire execution cluster of the Intel Core 2 Duo and Core i7 processors [26, 42]. Motorola has also used STE extensively

Tom Melham
Department of Computer Science, University of Oxford, Wolfson Building, Parks Road, Oxford, OX1 3QD, UK. e-mail: Tom.Melham@cs.ox.ac.uk

for verification of embedded custom memories [46], and has developed a suite of advanced methodologies and tools to support this task [72, 10, 11].

In the abstraction lattice at the heart of STE, each circuit node (i.e. wire carrying a single bit) is assigned a value in the set $\{\mathrm{X}, 0, 1\}$, with 'X' representing an 'unknown' value. An assignment of such values to every circuit node is an abstraction of a set of Boolean circuit states. It is abstract in the sense that it ambiguously stands for any one of a family of Boolean state sets, one for each replacement of every X by 0 or 1. The collection of all such abstractions forms a lattice, ordered by the amount of information we have about node values.

The STE model-checking algorithm uses three-valued circuit simulation [14] to compute a reachable abstract state-set in this representation. The algorithm is space-efficient because it operates over abstractions of sets of states; any parts of the circuit function not relevant to the specification get 'abstracted away' to X. Any correctness result verified in this abstract model transfers over to the real, Boolean model of circuit states. Formally, there is a Galois connection between the three-valued model and the Boolean model of states [19].

This abstraction machinery is controlled by the way in which a user writes properties for model checking. By careful coding of the property, the user can guide the symbolic simulation done during model-checking through the right layers of the abstract state lattice to verify the property with contained complexity. A good illustration of success is the content-addressable memory verification done by Pandey and colleagues [53], in which a careful encoding of properties gives a logarithmic reduction in complexity.

Specifications themselves are written in a very simple linear-time temporal logic, limited to implications between formulas with only conjunction and the next-time operator. STE specifications therefore express only bounded time properties, and it may take more properties to verify the same functionality in STE than in, say, CTL—the approach is in some ways similar to the FSM decomposition methodology of Interval Property Checking [51]. Different versions and extensions of STE with more expressive logics exist [32, 36, 64, 78]. But the simple form is the most widely tested on industrial applications and the focus of this chapter.

Although the logic of STE seems weak, its expressive power and abstraction capability are greatly enhanced by use of *symbolic* ternary simulation [16]. On top of the abstraction lattice, STE provides a layer of symbolic representation whereby whole *families* of abstractions, each covering only a part of the circuit's function, may be checked simultaneously. This mechanism, sometimes called 'symbolic indexing', provides a flexible way to achieve case-splitting by data abstraction scheme. A characteristic example is a memory verification, in which an n-element memory is verified with an indexed family of n abstractions, one for each address at which some target data might be located.

In STE, the cases covered by the abstractions in an indexed family can overlap, in contrast to existential abstraction for CTL [22, 28], which partitions the state space. Symbolic indexing can also record inter-dependencies among node values, and so increases the expressive power of specifications for STE. For example, input/output functions can be extracted from a circuit by using symbolic simulation to derive for-

mulas for the values on output nodes as functions of variables standing for arbitrary values on input nodes. These can then be checked against a specification in the form of some reference formulas provided by the verification engineer. Disjunction can also be expressed.

This chapter explains the theory of STE model checking in detail and briefly describes how the method is implemented and used in practice. It also gives a brief sketch of some extensions to STE, including Generalized Symbolic Trajectory Evaluation (GSTE). A full account of the theory of STE by its originators, Carl Seger and Randy Bryant, can be found in [64]. An illuminating perspective on the foundations of STE is provided by Chou in [19]. There is an in-depth tutorial on STE by Seger and Hazelhurst [32]. Roorda and Claessen also provide a tutorial account of STE in [20] and discuss its semantics in [55]. A usage methodology for STE in industrial practice can be found in [1, 39, 65]. GSTE is introduced and explained in [21, 68, 69, 78], and its relationship to conventional symbolic model checking is explored in [60].

25.2 Notational Preliminaries

We write $\triangleq$ to mean *equals by definition*. We assume familiarity with elementary propositional logic and predicate calculus notation and use the symbol $\supset$ for logical implication. We denote the set of Boolean truth-values by $\mathbb{B} = \{\mathrm{T}, \mathrm{F}\}$. We use lower-case letters (e.g. a, v, x, y_1) as Boolean variables, and use upper-case letters (e.g. P, Q) to stand for formulas of propositional logic ('Boolean functions'). We write $\vec{x}$ to mean a vector of unique variables $x_1, \ldots, x_n$ for indeterminate n and similarly $\vec{P}$ to stand for a vector of formulas $P_1, \ldots, P_n$.

The notation $P[\vec{Q}/\vec{x}]$ stands for the result of simultaneously substituting the formulas $\vec{Q}$ for all occurrences of the respective Boolean variables $\vec{x}$ in P. The notation $P[\vec{x}]$ should be taken to mean a formula that may contain free occurrences of the distinct Boolean variables $\vec{x}$. In a context in which a formula has been written $P[\vec{x}]$, subsequent use of the notation $P[\vec{Q}]$ can then be understood to mean the result of substituting the formulas $\vec{Q}$ for the variables $\vec{x}$ in $P[\vec{x}]$.

If P is a propositional formula and ϕ is a function that assigns a truth-value to each Boolean variable in it, we write $\phi \Vdash P$ to mean that ϕ satisfies P [57].

We also assume familiarity with the notation of naive set theory [29] and the rudiments of conventional functional programming notation for higher-order functions [12]. If A and B are sets, we write $A \to B$ for the set of all total functions from A to B. We assume that $\to$ associates to the right, so $A \to (B \to C)$ may be written $A \to B \to C$. We write function applications by mere juxtaposition: $f\ x$ instead of $f(x)$. For higher-order functions, function application associates to the left; so if $f \in A \to B \to C$, $a \in A$, and $b \in B$, then we can write $f\ a\ b$ for $(f\ a)\ b$.

The semantics of symbolic trajectory evaluation uses some elementary concepts of lattice theory [24]. A *poset* $(S, \sqsubseteq)$ is a partial order $\sqsubseteq$ on a set S. If $(S, \sqsubseteq)$ is a poset and $A \subseteq S$, then $x \in S$ is an *upper bound* for A exactly when $a \sqsubseteq x$ for all $a \in A$.

A *lower bound* is defined dually. An upper bound x of A is the *least upper bound* of A, written $\sqcup A$, if $x \sqsubseteq y$ for every upper bound y of A. The *greatest lower bound*, written $\sqcap A$, is defined dually. We also write $a \sqcup b$ (read 'a join b') for $\sqcup\{a,b\}$ when it exists and $a \sqcap b$ (read 'a meet b') for $\sqcap\{a,b\}$ when it exists.

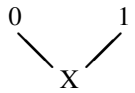
A poset $(S, \sqsubseteq)$ is a *complete lattice* iff $\sqcup A$ and $\sqcap A$ exist for all $A \subseteq S$. If S is finite and $a \sqcup b$ and $a \sqcap b$ exist for all $a, b \in S$, then $(S, \sqsubseteq)$ is a complete lattice.

25.3 Sequential Circuit Models in STE

We suppose there is a finite set of circuit nodes $\mathscr{N}$, naming observable points in circuits. You can think of a node as the name of a wire—i.e. just an identifier, such as 'reset' or '$\mathsf{input32}$'. As usual for sequential circuit models, there is a node in $\mathscr{N}$ to name each primary circuit input, as well as a node to name the output of each latch or other state-holding element. In STE, we can also give names to selected internal wires within a block of combinational logic, if we wish to express verification properties that make reference to values on these wires. In the interest of clarity, however, we will ignore this possibility for now.

25.3.1 States and Sequences

To describe the behavior of a circuit formally, we need to say which succession of values will be present on each of its nodes as the circuit evolves over time. Symbolic trajectory evaluation employs a three-valued state model, with values drawn from the set $\mathscr{D} = \{\mathrm{X}, 0, 1\}$. The usual binary values 0 and 1 are augmented with an additional value X. This stands for an unknown, which we represent mathematically by a partial order $\leq$ on $\mathscr{D}$, in which $\mathrm{X} \leq 0$ and $\mathrm{X} \leq 1$:



This orders values by information content: X stands for an unknown value and so is ordered below 0 and 1.

A *state* is an instantaneous snapshot of circuit behavior given by an assignment of values in $\mathscr{D}$ to all the node names in $\mathscr{N}$. A state is represented mathematically by a function

$$s \in \mathscr{N} \to \mathscr{D}$$

that maps each node name to a value.

If the set of circuit nodes is small enough, we can write down specific states just by giving the function explicitly. For example, for $\mathscr{N} = \{\mathsf{a}, \mathsf{b}, \mathsf{c}\}$, we might write

$$\{(\mathsf{a},0), (\mathsf{b},\mathrm{X}), (\mathsf{c},1)\} \qquad \text{or} \qquad \{\mathsf{a} \mapsto 0, \mathsf{b} \mapsto \mathrm{X}, \mathsf{c} \mapsto 1\}$$

for the state in which a is 0, b is X, and c is 1. We can use a more compact notation if we give an ordering on nodes. If the nodes in this example are ordered $\mathsf{a} < \mathsf{b} < \mathsf{c}$, we can just write '0X1' to denote this state. In what follows, we will feel free to use either of these notations, as appropriate.

The ordering $\leq$ on $\mathscr{D}$ is extended point-wise to get an ordering $\sqsubseteq$ on states. For reasons that will be clear later, we wish this to form a complete lattice, and so we will use a special symbol, $\top$, for the top element of this state lattice. We then define the set of states $\mathscr{S}$ to be $(\mathscr{N} \to \mathscr{D}) \cup \{\top\}$. The required ordering is defined for states $s_1, s_2 \in \mathscr{S}$ as follows:

$$s_1 \sqsubseteq s_2 \triangleq \begin{cases} s_2 = \top, \text{or} \\ s_1, s_2 \in \mathscr{N} \to \mathscr{D} \text{ and } s_1(n) \leq s_2(n) \text{ for all } n \in \mathscr{N} \end{cases}$$

The intuition is that if $s_1 \sqsubseteq s_2$, then s_1 may have 'less information' about node values than s_2—i.e. it may have Xs in place of certain 0s and 1s.

25.3.2 Abstraction

The term *state* just introduced is really a misnomer: a 'state' in STE, such as 0X1, is really an abstraction—an *abstract predicate* on sets of actual circuit states. Real circuit states are always Boolean-valued: each circuit node is either 0 or 1 and there isn't really a *value* 'X'. Roughly speaking, you can think of a lattice state as standing for a set of ordinary, Boolean-valued states. More precisely, a single lattice state stands (ambiguously, because it is an abstraction) for any one of a certain group of structurally-related sets of Boolean states.

Computational manipulation of sets of circuit states is a key idea in classical model checking. STE model checking also operates over sets of Boolean circuit states, but there is abstraction: the sets are only incompletely known or specified. This abstraction scheme is related to the *Cartesian abstraction* employed in some approaches to software model checking [6].

Suppose we have only two circuit nodes, a and b. And for the purpose of compactly writing down lattice states, we order them $\mathsf{a} < \mathsf{b}$. The lattice state 0X can be seen as an abstraction of the set of Boolean-valued states

$$\{\{\mathsf{a} \mapsto 0, \mathsf{b} \mapsto 0\}, \{\mathsf{a} \mapsto 0, \mathsf{b} \mapsto 1\}\}.$$

In the abstract 'state' 0X the node b is assigned the unknown value X. In the concrete set of Boolean states, there is one state in which b is 0 and one in which b is 1. So the node b can have either value. In STE, the lattice state 0X is also an abstraction of the following singleton set of Boolean-valued states:

$$\{\{\mathsf{a} \mapsto 0, \mathsf{b} \mapsto 0\}\}$$

Here, node b must be 0. Finally, the lattice state 0X is also an abstraction of the set of Boolean states

$$\{\{\mathsf{a} \mapsto 0, \mathsf{b} \mapsto 1\}\},$$

in which node b must be 1. The lattice-valued 'state' 0X is an abstraction of all three of these sets of real circuit states. That is, there is loss of information, the defining characteristic of being an 'abstraction'.

In practice, you can often think informally about a lattice state s as standing for the set of Boolean states obtainable by replacing all occurrences of X in s by a combination of 0s and 1s in every possible way. Every set of Boolean states of which s is an abstraction is some subset of this 'maximal' set of Boolean states. A characteristic of this abstraction is that it ignores information about dependencies between node values. Consider, for example, the set of states $\{\{\mathsf{a} \mapsto 0, \mathsf{b} \mapsto 1\}, \{\mathsf{a} \mapsto 1, \mathsf{b} \mapsto 0\}\}$, in which a and b always have values that are the negation of each other. The only abstraction of this set of states is XX.

One can view abstract 'states' s_1 and s_2 as *constraints* or *predicates* on sets of actual states of the hardware. If $s_1 \sqsubseteq s_2$, then every set of Boolean states that satisfies s_2 also satisfies s_1. For example, suppose we again have nodes $\mathsf{a} < \mathsf{b}$ and consider $0X \sqsubseteq 01$. Lattice state 01 unambiguously stands for the singleton set of Boolean states $\{\{\mathsf{a} \mapsto 0, \mathsf{b} \mapsto 1\}\}$, which also satisfies the 'constraint' 0X. More precisely, any set of states of which s_2 is an abstraction is a subset of any set of states of which s_1 is an abstraction. We say that s_1 is 'weaker than' s_2. (Strictly speaking, $\sqsubseteq$ is reflexive and we really mean 'no stronger than', but it is common to be a little inexact and just say 'weaker than'.) The top value $\top$ represents the unsatisfiable constraint. The *join* operator on pairs of states in the lattice is denoted by '$\sqcup$'.

25.3.3 Time-dependent Behaviour

To model dynamic behavior, we represent time by the natural numbers $\mathbb{N}$. A sequence of states that the circuit passes through as it evolves over time is then represented by a function from time to states:

$$\sigma \in \mathbb{N} \to \mathscr{S}$$

Such a function is called a *sequence*. A sequence (that never produces the over-constrained top state $\top$) just assigns a value in $\{X, 0, 1\}$ to each circuit node at each point in time. For example, $\sigma\ 3\ \mathsf{reset}$ would be the value assigned to the reset node at time 3.

The ordering on states is extended point-wise to sequences in the usual way:

$$\sigma_1 \sqsubseteq \sigma_2 \quad \triangleq \quad \sigma_1(t) \sqsubseteq \sigma_2(t) \text{ for all } t \in \mathbb{N}$$

If $\sigma_1 \sqsubseteq \sigma_2$, then we say that the sequence σ_1 is 'weaker than' the sequence σ_2. As before, it would be more accurate to say 'no stronger than'.

We now introduce an operation on sequences that is used later in stating the semantics of STE. For any $i \geq 0$, the ith *suffix* of a sequence σ is written σ^i and defined by

$$\sigma^i\, t \stackrel{\triangle}{=} \sigma\,(t{+}i) \text{ for all } t \in \mathbb{N}.$$

Taking the ith suffix just shifts the sequence σ forward i points in time, discarding the first i states.

25.3.4 The Next State Function and Trajectories

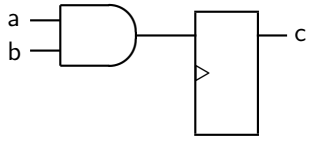
In symbolic trajectory evaluation, the formal model of a circuit c is given by a next-state function Y_c that maps states to states:

$$\mathrm{Y}_c \in \mathscr{S} \to \mathscr{S}$$

The subscript in Y_c identifies the particular circuit of interest; you can think of 'c' as a constant that names the circuit. This subscript is frequently omitted, as it is usually clear or doesn't matter which circuit is being discussed.

Intuitively, the next-state function expresses a constraint on the set of possible Boolean states into which the circuit may go for any set of Boolean states it might currently be in. Suppose the circuit is in abstract state $s \in \mathscr{S}$. Then $\mathrm{Y}\,s$ is the most specified abstract state that covers all the sets of states the circuit can make a transition to from any set of states that satisfies s. Roughly speaking, if the only possible value for a circuit node n in every next state is one of 0 or 1, then $\mathrm{Y}\ s$ will assign that value to n. Otherwise n will be X in the next abstract state.

A small example is this trivial unit-delay AND-gate:



This circuit has three nodes, which for the purpose of writing down states we order $\mathsf{a} < \mathsf{b} < \mathsf{c}$. Suppose the current state is 111, i.e. all nodes are known to be high. Then the next state $\mathrm{Y}(111)$ will be XX1. Similarly, if the current state is 110, then the next state $\mathrm{Y}(110)$ will also be XX1. In fact, the next value of c doesn't depend on its value in the current state, so $\mathrm{Y}(11\mathrm{X}) = \mathrm{XX1}$ as well. In all these cases, we see that in the next state a and b are both X because they are primary inputs—they are 'non-deterministic'. If b is 0 in the current state, then c is going to be 0 in the next state, regardless of the value of a in the current state. Hence $\mathrm{Y}(\mathrm{X0X}) = \mathrm{XX0}$. Finally, we sometimes have insufficient information to determine the value of the output. If the current state is X1X, for example, then we don't know whether c is going to be 0 or 1. It may be either, and hence $\mathrm{Y}(\mathrm{X1X}) = \mathrm{XXX}$.

In essence, X serves as a 'don't care' (or, depending on context, 'don't know') value in STE reachable-state calculations. This allows an STE model checker to prune away parts of circuit function that have no effect on the properties being validated—a kind of semantic 'cone of influence' reduction [23]. On the other hand, if not enough is known about initial states of the system, Xs can permeate the computations enough to make the satisfaction of target properties indeterminate. We discuss this issue later.

A requirement for trajectory evaluation is that the next-state function Y is monotonic. That is, for all states s_1 and s_2

$$s_1 \sqsubseteq s_2 \text{ implies } \mathrm{Y}\ s_1 \sqsubseteq \mathrm{Y}\ s_2.$$

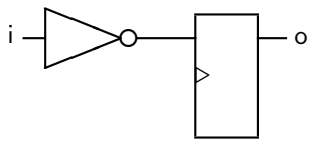
This is consistent with the idea that $\sqsubseteq$ is an information ordering: any increase in knowledge about the range of possible current states should produce only an increase—not a reduction or revision—of knowledge about the range of possible next states. Implementations of STE are designed to extract a next-state function that has this property from the circuit under analysis. This condition can be met, with some careful engineering, for a wide variety of common circuit design styles, including synchronous systems with latches as well as flip-flops, and systems with gated clocks. Transistor switch-level models can also be devised for STE [32, 64].

For a circuit c, we define the set of all its *trajectories*, $\mathscr{T}_c$, as follows:

$$\mathscr{T}_c \triangleq \{\sigma \mid \mathrm{Y}_c(\sigma\, t) \sqsubseteq \sigma\ (t{+}1) \text{ for all } t \in \mathbb{N}\}$$

For a sequence σ to be a trajectory, each successor state in the sequence must be at least as specified (and not contradict) the result of applying the next-state function Y_c to the previous state in the sequence. This ensures that σ is consistent with the circuit model Y_c, i.e. that the succession of abstract state constraints given by σ does not contradict what the hardware would actually do.

For example, suppose $\mathscr{N} = \{\mathsf{i}, \mathsf{o}\}$ with $\mathsf{i} < \mathsf{o}$ and consider the circuit



The partial sequence $\sigma = 1\mathrm{X}, \mathrm{XX}, \ldots$ does not begin a trajectory. In the second state of circuit execution we 'know' that o must be 0, because $\mathrm{Y}_c(1\mathrm{X}) = \mathrm{X0}$. But the value of o is unconstrained by the second state, XX, in the sequence σ. On the other hand, a trajectory can still over-approximate real circuit execution. For example, the partial sequence $\sigma = \mathrm{XX}, \mathrm{X0}, \ldots$ can begin a trajectory, because $\mathrm{Y}_c(\mathrm{XX}) = \mathrm{XX} \sqsubseteq \mathrm{X1}$. But this sequence does not constrain node i to be 0 in the first state. Of course, any sequence completely free of Xs is a trajectory exactly when it represents an actual Boolean execution trace of the circuit being modelled.

25.4 Trajectory Evaluation Logic

One of the keys to the efficiency of STE and its success with data-path circuits is its restricted temporal logic, which we now define. Certain formulas in this logic contain, as subformulas, expressions of ordinary propositional logic with (free) Boolean variables. Let P range over propositional formulas whose free variables are drawn from some set $\mathscr{V}$. A *trajectory formula* is a simple linear-time temporal logic formula with the following syntax:

$$\begin{array}{lll} f := & n \text{ is } 0 & \text{- node } n \text{ has value } 0 \\ \mid & n \text{ is } 1 & \text{- node } n \text{ has value } 1 \\ \mid & f \text{ and } g & \text{- conjunction of formulas} \\ \mid & \mathsf{N}\, f & \text{- } f \text{ holds in the next time step} \\ \mid & P \rightarrowtriangle f & \text{- } f \text{ is asserted only when } P \text{ is true} \end{array}$$

where f and g range over formulas and $n \in \mathscr{N}$ ranges over the nodes of the circuit. The formula P is called a *guard* and may contain propositional variables in $\mathscr{V}$. The role of these variables will be discussed later—for now, it is helpful to note that they are distinct from the names of circuit nodes in the set $\mathscr{N}$.

The basic trajectory formulas 'n is 0' and 'n is 1' say that the circuit node n has value 0 or value 1, respectively. The operator and forms the conjunction of trajectory formulas. The trajectory formula $\mathsf{N}\, f$ says that the trajectory formula f holds at the next point of time. It is easy to see that any formula containing only these first four syntactic constructs simply asserts the presence of 0 or 1 at certain fixed points of time on the specified circuit nodes. For example,

$$\mathsf{clk} \text{ is } 0 \text{ and } \mathsf{N}(\mathsf{clk} \text{ is } 1 \text{ and } \mathsf{N}(\mathsf{clk} \text{ is } 0 \text{ and } \mathsf{N}(\mathsf{clk} \text{ is } 1)))$$

describes two cycles of a clock clk that starts off low. It is obvious that the next-time operator distributes over conjunction.

The final construct $P \rightarrowtriangle f$ weakens the sub-formula f by requiring it to be satisfied only when the guard P is true. In essence, a trajectory formula represents a whole *set* of assertions about the presence of the Boolean values 0 and 1 on particular circuit nodes over time. A guard is a propositional formula that may contain Boolean variables, and a trajectory formula $P \rightarrowtriangle f$ with a guard P asserts f only for satisfying assignments of values to the Boolean variables in P. So for any trajectory formula, each assignment of values to the variables in all its guards gives a (possibly different) assertion about the presence of 0s and 1s on certain circuit nodes at particular points in time.

A trivial example is this trajectory formula:

$$x \rightarrowtriangle (\mathsf{a} \text{ is } 0 \text{ and } \mathsf{b} \text{ is } 1) \;\text{ and }\; \bar{x} \rightarrowtriangle (\mathsf{a} \text{ is } 1 \text{ and } \mathsf{b} \text{ is } 0)$$

The guards here are just the literals x and $\bar{x}$. The formula encodes two of the four possible input bit patterns you might present to the unit-delay AND-gate in Sect. 25.3.4,

namely the ones in which a and b are mutex. More generally, the guards in a trajectory formula can be any propositional logic expressions and can have variables in common. This gives STE the expressive power to represent inter-dependencies among node values—and, as will be seen, to encode families of state abstractions for efficient model checking.

We can associate an arbitrary propositional formula P with a circuit node using the construct 'n is P' defined by

$$n \text{ is } P \stackrel{\triangle}{=} (P \rightarrow n \text{ is } 1) \text{ and } (\overline{P} \rightarrow n \text{ is } 0)$$

We suppose that $\rightarrow$ binds more tightly than and, so may drop the brackets:

$$n \text{ is } P \stackrel{\triangle}{=} P \rightarrow n \text{ is } 1 \text{ and } \overline{P} \rightarrow n \text{ is } 0$$

In the simplest case, we use this construction to attach distinct Boolean variables to input nodes for direct symbolic simulation of circuits. For example, we might assert the formula 'a is x and b is y' for our unit-delay AND-gate. Here, the variables x and y just name the values present on the input nodes a and b. We might expect, after simulation, the circuit model to satisfy '$\mathsf{N}(\mathsf{c} \text{ is } x \wedge y)$'.

25.4.1 Correctness Properties for Verification

Circuit correctness in symbolic trajectory evaluation is stated by *trajectory assertions* of the form $A \Rightarrow C$, where A and C are trajectory formulas. The intuition is that the *antecedent* A provides stimuli to circuit nodes and the *consequent* C specifies the values expected on circuit nodes as a response, after simulation. We might, for example, write

$$\mathsf{a} \text{ is } x \text{ and } \mathsf{b} \text{ is } y \;\Rightarrow\; \mathsf{N}(\mathsf{c} \text{ is } x \wedge y)$$

for a direct, exhaustive simulation of our little AND-gate. Using guards, this single property encodes all the four possible bit-patterns that might be presented on the two inputs a and b. To check just the mutex cases mentioned earlier, we write

$$x \rightarrow (\mathsf{a} \text{ is } 0 \text{ and } \mathsf{b} \text{ is } 1) \;\text{ and }\; \bar{x} \rightarrow (\mathsf{a} \text{ is } 1 \text{ and } \mathsf{b} \text{ is } 0) \;\Rightarrow\; \mathsf{N}(\mathsf{c} \text{ is } 0)$$

Here there is not such a direct correspondence between 'Boolean variables' and 'values on circuit nodes'. The variable x doesn't correspond to an input value at all, but just enumerates the two input stimuli we wish to consider. In fact, because x doesn't appear in the consequent, this formula effectively expresses a disjunction in the antecedent. It is essential for a full understanding of STE to bear in mind that the Boolean variables used in guards, in the general case, need not correspond to node values. They are a case-enumeration mechanism, situated a layer above circuit values.

We now say what it means for a sequence σ to entail a trajectory formula f. Entailment is defined with respect to an assignment of Boolean truth-values to the Boolean variables in the guards of the formula. Suppose $\mathscr{V}$ is the set of all such variables and $\phi \in \mathscr{V} \to \{\mathrm{T},\mathrm{F}\}$ is an assignment of values to them. Define

$$\begin{array}{lcl}
\sigma \models_\phi n \text{ is } 0 & \triangleq & \begin{cases} \sigma(0) = \top, \text{ or} \\ \sigma(0) \in \mathscr{N} \to \mathscr{D} \text{ and } \sigma\, 0\, n = 0 \end{cases} \\
\sigma \models_\phi n \text{ is } 1 & \triangleq & \begin{cases} \sigma(0) = \top, \text{ or} \\ \sigma(0) \in \mathscr{N} \to \mathscr{D} \text{ and } \sigma\, 0\, n = 1 \end{cases} \\
\sigma \models_\phi f \text{ and } g & \triangleq & \sigma \models_\phi f \text{ and } \sigma \models_\phi g \\
\sigma \models_\phi \mathsf{N} f & \triangleq & \sigma^1 \models_\phi f \\
\sigma \models_\phi P \rightarrowtriangle f & \triangleq & \phi \Vdash P \text{ implies } \sigma \models_\phi f
\end{array}$$

where $\phi \Vdash P$ means that the propositional formula P is satisfied by the assignment ϕ of truth-values to the Boolean variables in P. It follows that if $\sigma_1 \models_\phi f$ and $\sigma_1 \sqsubseteq \sigma_2$ then $\sigma_2 \models_\phi f$. That is, if a sequence σ_1 entails f, then any sequence σ_2 consistent with σ_1 but having 'more information' about node values also entails f. Informally, you can think of both trajectory formulas and sequences as predicates on sets of circuit states. And both $\models_\phi$ and $\sqsubseteq$ (backwards) as forms of entailment.

A trajectory assertion $A \Rightarrow C$ is true for a given ϕ exactly when every trajectory of the circuit that entails A also entails C. For a given circuit c, define $\models_\phi A \Rightarrow C$ to mean that for all $\sigma \in \mathscr{T}_c$, if $\sigma \models_\phi A$ then $\sigma \models_\phi C$. Intuitively, saying $\sigma \in \mathscr{T}_c$ means σ is an abstraction (over-approximation) of a set of actual execution traces of the circuit. For a given ϕ, the antecedent A describes a certain finite scattering of 0s and 1s across selected circuit nodes over a bounded period of time. Every trajectory $\sigma \in \mathscr{T}_c$ that conforms to this description must also exhibit the scattering of 0s and 1s expected by the consequent C.

The use of propositional formulas as guards enables a single trajectory assertion $A \Rightarrow C$ to encode a whole family of stimulus-response correctness properties, one for each ϕ. An example is the partial correctness specification for the unit-delay AND-gate discussed earlier:

$$x \rightarrowtriangle (\mathsf{a} \text{ is } 0 \text{ and } \mathsf{b} \text{ is } 1) \text{ and } \bar{x} \rightarrowtriangle (\mathsf{a} \text{ is } 1 \text{ and } \mathsf{b} \text{ is } 0) \Rightarrow \mathsf{N}(\mathsf{c} \text{ is } 0)$$

When $\phi(x) = \mathrm{T}$, this property enforces correctness for the specific case where the input values are $\mathsf{a} = 0$ and $\mathsf{b} = 1$. When $\phi(x) = \mathrm{F}$, it enforces correctness for the case where $\mathsf{a} = 1$ and $\mathsf{b} = 0$. As will be seen, the model checking algorithm for STE treats this dependence on ϕ symbolically, verifying all the cases simultaneously. We write $\models A \Rightarrow C$ to mean that $\models_\phi A \Rightarrow C$ holds for all ϕ.

It can be seen from the semantics just given that STE is fundamentally a logic of *forward* propagation of information about circuit values over time. One might hope the following trajectory assertion would hold of the little unit-delay AND-gate introduced in Sect. 25.3.4:

$$\mathsf{N}(\mathsf{c} \text{ is } 1) \Rightarrow \mathsf{a} \text{ is } 1 \text{ and } \mathsf{b} \text{ is } 1$$

The output c cannot possibly be high, unless both inputs a and b were high on the last clock cycle. Observe, however, that $\sigma = \mathrm{XXX}, \mathrm{XX1}, \mathrm{XXX}, \mathrm{XXX}, \ldots$ is a trajectory of this circuit that satisfies the antecedent but not the consequent. So this property is false. A sketch of what it would take to have the semantics of a 'bidirectional STE' in which this property could hold is given by Roorda in [54]. The fundamentally forwards nature of the STE logic flows from its basis as a logic of (forwards) circuit simulation. Roorda and Claessen have proposed to use SAT to encode the circuit along with the verification in an STE model-checking run, allowing backward information propagation.

We also note in passing that it is possible for an antecedent to be unsatisfiable, either outright or by all (or just some) real circuit executions. This is the problem of *antecedent failure* [7] or, more generally, *vacuity* in logic-based verification [8]. The most obvious case is where the antecedent places inconsistent values on some circuit node, as for example in

$$x \rightarrowtriangle \mathsf{a\ is\ 0\ and\ } y \rightarrowtriangle \mathsf{a\ is\ 1}.$$

This is inconsistent when ϕ assigns the same truth-value to x and y. Another case is when the antecedent places a value on a circuit node that disagrees with what circuit execution itself produces. (For this to happen, the node in question must be the output of a state-holding element, not a primary input.) The semantic representation for the circuit state arising in the presence of such an inconsistency is the special top state, $\top$, introduced in Sect. 25.3.1.

Both types of antecedent failure can be detected by STE implementations and an error raised or warning given. Typically, a practitioner will then investigate the cause and rewrite the offending antecedent. Sometimes antecedent failure can safely, if cautiously, be ignored because it is known that cases in which inconsistency occurs do not to arise in practice. A more subtle problem is so-called *hidden vacuity* [70]. This can occur when the antecedent requires some specific value, 1 say, on a particular node that in any actual circuit execution would be 0, but which is assigned X by the circuit model because of incomplete information. For a full analysis and some methods to detect vacuity in STE, including the hidden type, see [70].

25.4.2 Correctness Properties under Assumptions

It is convenient to have a notation that restricts the valuations ϕ for which a correctness property $\models_\phi A \Rightarrow C$ is verified to those that satisfy some given predicate. If P is a formula of propositional logic, we write $P \vdash A \Rightarrow C$ to mean that $\models_\phi A \Rightarrow C$ holds for all ϕ that satisfy P. This is used extensively in practice to express environmental constraints or assumptions on input values, represented directly by Boolean variables. The verification script for an industrial scale sub-circuit may well have hundreds of lines of text generating such formulas in order to accurately characterize the complex environment in which it operates.

Note that antecedents alone can also encode assumptions: writing the antecedent ‘a is x and b is x’ makes the implicit assumption that two input nodes a and b have the same value. A methodology used by Intel for structuring both kinds of assumptions and translating them into System Verilog Assertions is presented in [45]. This supports a form of assume-guarantee reasoning; blocks of circuitry are verified in STE under assumptions about the operating environment, which are then translated into System Verilog Assertions and checked with simulation.

25.4.3 Internal Combinational Circuitry

It remains to note that what real implementations of STE do is not fully reflected in the simple story presented above. We have assumed throughout, and in particular in our formulation of next-state functions, that models and formulas refer only to circuit nodes that are primary inputs or outputs of state-holding elements. But, as mentioned earlier, STE trajectory formulas can also mention internal nodes of combinational logic. A precise formulation of the semantics of this is a little involved [55], but the intuition is roughly the following. Suppose n is an internal node, driven by some fanin cone of combinational logic. The sub-formula ‘n is 1’ should hold exactly when forward propagation through this logic of all known information about other node values in the circuit implies that n must have value 1. The practical value of this capability is that it exposes the fine detail of internal values within complex circuits to inspection and debugging by STE model checking.

25.5 The Fundamental Theorem of Trajectory Evaluation

A key property of trajectory evaluation logic is that for any trajectory formula f there exists a unique weakest sequence that entails f. (Assuming a fixed assignment ϕ of Boolean values to variables.) This is called the *defining sequence* for f and is written $[f]^\phi$. It is defined by recursion over the syntax of trajectory formulas. Recall that a sequence is an element of $\mathbb{N} \to ((\mathscr{N} \to \mathscr{D}) \cup \{\top\})$; in the definition that follows, $t \in \mathbb{N}$ ranges over points in time and $n \in \mathscr{N}$ ranges over node names.

$$
\begin{array}{lll}
[m \text{ is } 0]^\phi \; t\, n & \triangleq & 0 \text{ if } m{=}n \text{ and } t{=}0, \text{ otherwise X} \\
[m \text{ is } 1]^\phi \; t\, n & \triangleq & 1 \text{ if } m{=}n \text{ and } t{=}0, \text{ otherwise X} \\
[f \text{ and } g]^\phi \; t & \triangleq & ([f]^\phi \; t) \sqcup ([g]^\phi \; t) \\
[\mathsf{N}\, f]^\phi \; t\, n & \triangleq & [f]^\phi \; (t{-}1)\; n \text{ if } t{\neq}0, \text{ otherwise X} \\
[P \rightarrowtriangle f]^\phi \; t\, n & \triangleq & [f]^\phi \; t\; n \text{ if } \phi \Vdash P, \text{ otherwise X}
\end{array}
$$

Note that, in the clause for and, we take the lattice join ($\sqcup$) of states.

The crucial property enjoyed by this definition is that $[f]^\phi$ is the unique weakest sequence that satisfies f for the given ϕ. That is, for fixed ϕ and any trajectory σ,

we have that $\sigma \models_\phi f$ if and only if $[f]^\phi \sqsubseteq \sigma$. STE enjoys this property only because the language of trajectory formulas is monotonic—the defining sequence would not exist if the logic included negation or disjunction. (But see [32] for a more general presentation that does include negation, and a strong 'until' operator too.)

We can also define the weakest *trajectory* that satisfies each formula, for a given ϕ. Recall that a trajectory is a sequence of abstract states that respects the next-state function of the circuit—an abstract constraint on sequences of circuit states that is consistent with circuit function. The *defining trajectory* for a formula, written $[\![f]\!]^\phi$, is given by the following recursive calculation over time points:

$$\begin{array}{lcll} [\![f]\!]^\phi\ 0 & \triangleq & [f]^\phi\ 0 & (25.1) \\ [\![f]\!]^\phi\ (t{+}1) & \triangleq & [f]^\phi\ (t{+}1) \sqcup \mathrm{Y}_c([\![f]\!]^\phi\ t) & \end{array}$$

The defining trajectory of a formula f is its defining sequence with the added constraints on state transitions imposed by the circuit, as modelled by the next-state function Y_c. The first state $[\![f]\!]^\phi\ 0$ is just given by any assumptions on node values at time 0 made by the antecedent. (Hence, in STE model checking, the initial state of a circuit is completely unknown by default. If you want to start in some specified initial state space, you have to characterize this explicitly in the antecedent.) Each successive state is obtained by combining, using the lattice join operator, any assumptions about node values at that time made by the antecedent with the constraints given by applying the model to the previous state. It is easy to see that folding in the effect of Y_c ensures that the sequence $[\![f]\!]^\phi$ is indeed a *trajectory* of the circuit c. Note also that if there is a conflict between the circuit and the antecedent at any point in time, then joining the two constraints will result in the top state $\top$.

It can be shown that $[\![f]\!]^\phi$ is the unique weakest trajectory that satisfies f, for the given ϕ. That is, for fixed ϕ and any trajectory $\sigma \in \mathscr{T}_c$, we have that $\sigma \models f$ if and only if $[\![f]\!]^\phi \sqsubseteq \sigma$.

The *Fundamental Theorem of Trajectory Evaluation* [64] follows immediately from the previously-stated properties of $[f]^\phi$ and $[\![f]\!]^\phi$. It states that for a given ϕ, the trajectory assertion $\models_\phi A \Rightarrow C$ holds exactly when $[C]^\phi \sqsubseteq [\![A]\!]^\phi$. The intuition is that the weakest trajectory satisfying the antecedent must entail the defining sequence characterizing the consequent: what the circuit 'does', under the assumptions made, must be among the things it is intended to do.

25.6 STE Model Checking

The Fundamental Theorem of Trajectory Evaluation gives a model-checking algorithm for trajectory assertions: to see if $\models_\phi A \Rightarrow C$ holds for a given ϕ, just compute the sequences of states $[C]^\phi$ and $[\![A]\!]^\phi$ and compare them point-wise for every circuit node at every relevant point in time. This works because both A and C will have only a finite number of nested next-time operators, and so only finite initial segments of the defining trajectory and defining sequence need to be calculated and compared.

In practice, the defining trajectory of A is computed iteratively, and each resulting state is checked against the consequent as it is generated. In essence, the algorithm does a step-by-step unrolling of the recursive definition of $[\![A]\!]^\phi$ shown above as Eq. (25.1). Throughout the computation, the current state is maintained as an assignment of a value from $\mathscr{D}$ to each circuit node. Initially, all nodes are set to X. Any nodes assumed to have specific values by the antecedent A at time 0 are then updated to have these values. Any constraints on nodes at time 0 in the consequent can then be checked against these initial node values. At each subsequent step, the algorithm first computes the next state of the circuit from the current one, according to the next-state function Y_c. This is done in practice by three-valued simulation of a circuit net-list description [14], in which logic gates or other primitive circuit elements are interpreted to operate over the domain $\{X, 0, 1\}$. It is here, at the heart of STE, that propagation of 'information' about states is enacted. For example, the simulator's truth-table for an AND-gate would be

$\wedge$	X	0	1
X	X	0	X
0	0	0	0
1	X	0	1

Notice that the output is known to be 0 when any input is 0, even if another input is X. With some ingenuity, three-valued interpretations for simulation can be given to circuits in a range of design styles and at various abstraction levels—including, notably, transistors at the 'switch' level [15, 64]. Once the next state has been obtained, it is combined with any assumptions about circuit values in the current step stated by the antecedent. This is done by taking the lattice join ($\sqcup$) of the values, node by node—effectively calculating the join of the next state obtained from Y_c and the corresponding state in the defining sequence of the antecedent. (If there is a clash of values on any node, then the result is $\top$ and we have an antecedent failure.) Finally, any constraints on particular nodes at this time step in the consequent are checked against the calculated node values. The process continues for as many time steps as the deepest nesting of next-time operators in the trajectory assertion. If at any point there is a conflict between the consequent and a node value in the states being calculated, the property has been falsified. If they always agree, the property has been proved.

As already noted, all information about node values in the 'initial state' of an STE verification comes from only the antecedent of the property checked: each node starts out X, unless the antecedent explicitly assigns 0 or 1. A subtle issue of large practical significance is that the verification engineer must choose which input and state nodes to 'drive' via the antecedent, in order to produce a determinate circuit response to the antecedent stimulus. If too few input nodes are driven, Xs propagate into the simulation and the result fails to satisfy the consequent. The appearance of X on a node where the consequent expects a 0 or 1 is known as a *weak disagreement*. In this case the property is neither falsified nor proved.

For complex or industrial-scale circuits, it can be quite hard to find a minimal set of input nodes that need to be stimulated by the antecedent to get a determi-

nate result—to eliminate weak disagreements. Doing this is important because it increases the abstraction level and hence tractability of the verification, especially in the symbolic algorithm discussed in Sect. 25.6.1. The Forte methodology has a whole phase of work for this activity, dubbed *wiggling* by Robert Jones, and the Forte environment has specialized computational tools to support it [1, 39]. Some research on automated methods for refining STE properties to eliminate unwanted Xs is reported in [18, 56, 70].

As briefly mentioned earlier, properties in full STE can refer to internal nodes of combinational logic, as well as primary inputs and the outputs of state-holding elements. In the simulation engine that underlies STE there is, therefore, a phase during which values asserted on internal nodes by antecedents are propagated through this logic, before the consequent is checked. This can involve computing fixed points, for certain models of some circuit design styles.

25.6.1 The Symbolic Model Checking Algorithm

The model checking algorithm just sketched requires ϕ to be supplied: given a fixed assignment ϕ of values to variables in the guards, we calculate and compare $[C]^\phi$ and $[\![A]\!]^\phi$ point-wise. But much of the power of STE comes from the key observation that it is not necessary to supply ϕ in advance; instead, the comparison can be computed for all possible valuations, i.e. parametrically in ϕ. The result is a constraint on ϕ—a formula called a *residual*—that gives precisely the condition on Boolean variables in the guards under which the trajectory assertion $A \Rightarrow C$ is verified. If the residual is T, the circuit satisfies all the properties encoded by this assertion. If it is F, the circuit satisfies none of them. Otherwise, the residual encodes the class of all variable assignments under which an STE trajectory assertion $A \Rightarrow C$ holds. In other words, a residual is the weakest constraint R such that $R \models A \Rightarrow C$, in the notation of Sect. 25.4.2. This provides an enormously valuable and flexible debugging aid because, with R in hand, a verification engineer can explore the whole counterexample space computationally. The set of all counterexamples is encoded by $\overline{R}$.

The symbolic STE model checking algorithm works as follows. At the level of basic data values in $\{\mathrm{X}, 0, 1\}$, the required computation is to show that

$$[C]^\phi\ t\ n \leq [\![A]\!]^\phi\ t\ n \tag{25.2}$$

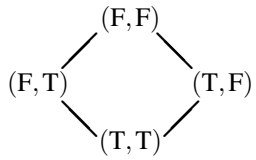
for all $t \geq 0$ and $n \in \mathscr{N}$. For each circuit node at each relevant point in time, we compare the value expected by the consequent to that given by the antecedent and circuit simulation. To make this comparison 'parametric' in ϕ, we use a pair of Boolean formulas to encode functions from ϕ to data values in $\mathscr{D}$ and do the comparison on this encoding.

The encoding is the following. First, enhance the three-valued set $\mathscr{D} = \{\mathrm{X}, 0, 1\}$ with an additional 'top' element, Z, that represents the value of an over-constrained node. This is not really a fundamental extension; any sensible implementation will

raise an exception if any node individually becomes Z, so any state with one or more nodes being Z in effect represents the top state $\top$. Now encode the four values $\{\mathrm{X},0,1,\mathrm{Z}\}$ by pairs of Booleans in $\mathbb{B}\times\mathbb{B}$, as follows:

$$\mathrm{X} \triangleq (\mathrm{T},\mathrm{T}) \qquad 0 \triangleq (\mathrm{F},\mathrm{T}) \qquad 1 \triangleq (\mathrm{T},\mathrm{F}) \qquad \mathrm{Z} \triangleq (\mathrm{F},\mathrm{F})$$

Then define an information ordering $\leq$ on these values to make it a lattice:



Formally, define $(a_1,a_2) \leq (b_1,b_2)$ to hold exactly when b_1 implies a_1 and b_2 implies a_2. It is easy to check that this definition captures the above Hasse diagram. The join of two elements is equally straightforward and is given by component-wise conjunction: $(a_1,a_2)\sqcup(b_1,b_2)$ equals (a_1 and b_1, a_2 and b_2).

This is the so-called 'dual-rail' encoding of four-valued logic employed in STE implementations [62]. It is straightforward to define logical operators over these values in terms of ordinary logical operations on the component Booleans. For example, the four-valued truth-table for conjunction is

$\wedge$	X	0	1	Z
X	X	0	X	0
0	0	0	0	0
1	X	0	1	Z
Z	0	0	Z	Z

which can be be defined by $(a_1,a_2)\wedge(b_1,b_2) \triangleq (a_1 \text{ and } b_1,\ a_2 \text{ or } b_2)$. As will be seen below, the underlying simulation engine of full symbolic STE interprets circuit net-lists in a symbolic version of this four-valued logic.

Now, a valuation is a function in $\mathscr{V}\to\mathbb{B}$, from variables $\mathscr{V}$ to the Boolean truth-values $\mathbb{B}$. We want the node values computed by our model checking algorithm to depend on a valuation, so the value associated with each node will now be a function from valuations to (encoded) lattice elements:

$$(\mathscr{V}\to\mathbb{B})\to(\mathbb{B}\times\mathbb{B})$$

Any such function f can be represented by a pair of formulas of propositional logic (P_1,P_2). Define P_1 so that it is satisfied by just those valuations that f maps to (T,a_2) for some a_2. Likewise, define P_2 so that it is satisfied by just those valuations that f maps to (a_1,T) for some a_1. In a nutshell, $(\mathscr{V}\to\mathbb{B})\to(\mathbb{B}\times\mathbb{B})$ is isomorphic to $((\mathscr{V}\to\mathbb{B})\to\mathbb{B})\times((\mathscr{V}\to\mathbb{B})\to\mathbb{B})$, and so a function from valuations to Booleans is straightforwardly represented by a pair of propositional formulas with free variables.

Given two functions f and g in this representation, we can compute a condition that characterizes the valuations ϕ for which $f\ \phi \leq g\ \phi$. Suppose f is represented by

the pair of formulas (P_1, P_2) and g is represented by the pair of formulas (Q_1, Q_2). Simply form the following formula of propositional logic:

$$(Q_1 \supset P_1) \wedge (Q_2 \supset P_2), \tag{25.3}$$

corresponding to the definition of the ordering $\leq$ just given. The result is a propositional formula satisfied precisely by the valuations ϕ for which $f\ \phi \leq g\ \phi$.

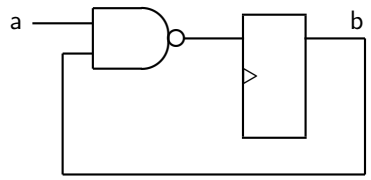
Using this idea, the symbolic version of STE proceeds just as indicated in Sect. 25.6, except that node values are now computed parametrically in the valuation. The algorithm maintains a pair of formulas for each circuit node, representing the value on each node as a function of an arbitrary valuation. Initially, every circuit node gets the dual-rail value (T, T), representing X for every valuation. Node values are then updated, step-by-step, to new symbolic dual-rail values according to the general scheme already explained. Note that values coming from the antecedent and consequent are now conditional on guards that constrain variables and so also represented by the dual-rail encoding. For example, if the antecedent says '$P \rightarrowtriangle \mathsf{n}\ \mathsf{is}\ 1$', this becomes the dual-rail value $(\mathrm{T}, \overline{P})$. Under any valuation that satisfies P, this equals (T, F), the encoding for 1; under any other valuation, it equals (T, T), the encoding for X. The circuit net-list is also interpreted over this dual-rail parametric representation of four-valued logic. For example, if (P_1, P_2) and (Q_1, Q_2) are the dual-rail values currently on the two inputs of an AND gate, then the output is given by the pair of formulas $(P_1 \wedge Q_1, P_2 \vee Q_2)$. Lattice joins are also computed symbolically—by component-wise conjunction, as before. Finally, whenever a comparison is made between a computed node value and a value specified by the consequent, a formula of propositional logic is built of the form shown in Eq. 25.3. The conjunction of all such formulas is the residual delivered as the final outcome of model-checking.

We note in passing that weak disagreements in this symbolic version of STE can be conditional on the valuation parametrically encoded in dual-rail values. That is, there may be a weak disagreement between the computed value of a node and the consequent's specification of its required value for only *some* valuations of the Boolean variables in play. This adds an extra dimension of complication and subtlety to the process of eliminating unwanted Xs and tools that support it.

In implementations of STE, the propositional formulas of the dual-rail encoding are represented by BDDs, or by some non-canonical representation such as AIGs (and-inverter graphs [34]). In the latter case, SAT technology can be used to check the final results of STE verification runs. Intel's Forte system has full support for both approaches.

25.6.2 Some Small Examples in Detail

We now illustrate the symbolic algorithm, in detail, by showing its calculations for a few properties of the following little example circuit:



Suppose we want to use STE to perform a full verification of this device by symbolic simulation. We introduce two Boolean variables, x and y, to stand for the initial values of the input node a and state node b, respectively. The trajectory assertion we want to check is

$$\mathsf{a\ is}\ x\ \mathsf{and\ b\ is}\ y \Rightarrow \mathsf{N}(\mathsf{b\ is}\ \overline{x} \vee \overline{y})$$

This employs a distinct, unconstrained Boolean variable to name the value on each node initially, and then checks that the expected function of these values appears on node b at the next cycle.

Model checking begins by assigning the dual-rail value $\mathrm{X} = (\mathrm{T},\mathrm{T})$ to all nodes, and then overlaying the assumptions made by the antecedent at time 0 onto this initial state. Recall that '$\mathsf{n\ is}\ P$' abbreviates '$P \rightarrowtriangle \mathsf{n\ is\ 1\ and}\ \overline{P} \rightarrowtriangle \mathsf{n\ is\ 0}$'. So the antecedent really makes two guarded assertions about node a, encoded by dual-rail values as follows:

$$x \rightarrowtriangle \mathsf{a\ is\ 1,\ encoded\ by}\ (\mathrm{T},\overline{x}) \qquad \text{and} \qquad \overline{x} \rightarrowtriangle \mathsf{a\ is\ 0,\ encoded\ by}\ (x,\mathrm{T}).$$

The semantics of trajectory formulas for and takes the lattice join of states, so the overall assumption made by the antecedent about node a is

$$(\mathrm{T},\overline{x}) \sqcup (x,\mathrm{T}) = (\mathrm{T} \wedge x, \overline{x} \wedge \mathrm{T}) = (x,\overline{x})$$

Likewise, the dual-rail encoding of the antecedent's assumptions about node b are given by $(y,\overline{y})$. State 0 of the defining trajectory for antecedent is therefore

node	value
a	$(x,\overline{x})$
b	$(y,\overline{y})$

Notice that neither of these nodes is ever X—for every valuation, you get either $(\mathrm{T},\mathrm{F}) = 1$ or $(\mathrm{F},\mathrm{T}) = 0$, so there is going to be no abstraction in this verification. In general, if the two 'rails' of a dual-rail value are the negation of each other, the encoded function on valuations never yields lattice values X or Z.

Now circuit simulation computes the state at time 1. The conjunction of values on a and b, according to the truth-table given earlier, is encoded by $(x \wedge y, \overline{x} \vee \overline{y})$. Negation of dual-rail values is done simply by swapping the components of the pair, so the state of the circuit at time 1 is

node	value
a	(T,T)
b	$(\overline{x}\vee\overline{y},x\wedge y)$

Notice that a has been reset to X, because it is a primary input and therefore free to take on any new value at each cycle. The antecedent makes no assumptions about values at time 1, so this is also the state at time 1 in the defining trajectory.

It remains to check this state against the expectations of the consequent—i.e. state 1 of the defining sequence of the consequent. It is easy to see that the dual-rail value that encodes 'b is $\overline{x}\vee\overline{y}$' exactly matches the computed value just calculated for b in state 1 of the defining trajectory. Plugging both values into Eq. (25.3) yields a tautology, so the residual is equivalent to T and the property is satisfied outright.

In this example, there is no abstraction and the property holds for all valuations. It is instructive to sketch—very briefly—examples that illustrate abstraction and a non-constant residual. For the first, consider the property

$$x \rightarrowtriangle \mathsf{a}\ \mathsf{is}\ 0\ \mathsf{and}\ \overline{x} \rightarrowtriangle \mathsf{b}\ \mathsf{is}\ 0 \Rightarrow \mathsf{N}(\mathsf{b}\ \mathsf{is}\ 1)$$

This verifies all the input cases of our little circuit for which the next state of b is 1. This time, the propositional variable x does not straightforwardly correspond to 'the value on an input', but serves as a symbolic 'index' to enumerate the two input stimuli we want to cover.

It is easy to see the first state of the defining trajectory for this example is

node	value
a	$(\overline{x},\mathrm{T})$
b	(x,T)

There is abstraction here. Node b is X for valuations that set x to true, and node a is X for valuations that set x to false. After four-valued symbolic simulation, state 1 of the defining trajectory is found to be

node	value
a	(T,T)
b	$(\mathrm{T}\vee\mathrm{T},\overline{x}\wedge x)$

The dual-rail encoding of values on node b simplifies to (T,F), which encodes the constant lattice value 1. This agrees with the consequent, and so the property holds.

To see a non-constant residual, consider the following property:

$$\mathsf{a}\ \mathsf{is}\ x\ \mathsf{and}\ \mathsf{b}\ \mathsf{is}\ y \Rightarrow \mathsf{N}(\mathsf{b}\ \mathsf{is}\ 1)$$

We have again 'driven' both nodes a and b with symbolic Boolean values x and y, respectively. As before, state 1 of the defining trajectory will be

node	value
a	(T,T)
b	$(\overline{x}\vee\overline{y},x\wedge y)$

To get the residual, we compare this, node-by-node, to state 1 of the consequent:

node	value
a	(T,T)
b	(T,F)

and conjoin the results. Calculating with the symbolic version of $\leq$ gives:

$$\text{Node a}: \quad (\mathrm{T},\mathrm{T}) \leq (\mathrm{T},\mathrm{T}) = (\mathrm{T} \supset \mathrm{T}) \wedge (\mathrm{T} \supset \mathrm{T}) = \mathrm{T}$$

$$\text{Node b}: \quad (\mathrm{T},\mathrm{T}) \leq (\mathrm{T},\mathrm{T}) = (\mathrm{T} \supset \overline{x}\vee\overline{y}) \wedge (\mathrm{F} \supset x\wedge y) = \overline{x}\vee\overline{y}$$

The overall residual is therefore '$\overline{x}\vee\overline{y}$', representing the precise set of cases for which the circuit satisfies the specification. It is easy to see this is right: node b will be 1 after one clock cycle exactly when at least one of a and b starts off 0.

25.6.3 Model Checking Properties under Assumptions

Few industrial verifications take place in isolation from environmental and other operating assumptions about the blocks of circuitry under inspection. In practice, the properties to be verified in STE are virtually always of the form $P \models A \Rightarrow C$, where P expresses some potentially complex assumptions about the environment the circuit will operate in. Typically, there are a number of primary inputs directly driven by distinct Boolean variables in the antecedent, and the formula P says that these values satisfy some relational assumptions [45]. For example, when verifying a floating-point arithmetic unit, it may (and, for correctness, probably must) be assumed that the vectors of Boolean variables representing the incoming operands conform to the format laid down by the IEEE 754 floating point standard. When verifying normal modes of operation, it will have to be assumed that any reset or test-scan inputs are tied to the inactive state. And so on. Constraints may also arise from case-splitting strategies that partition the input space into tractable sub-spaces [65].

Conditional verifications like these are efficiently handled in STE as follows. Suppose the assumption $P[x]$ constrains some Boolean variables $\vec{x} = x_1,\ldots,x_n$ that occur in a trajectory assertion $A[\vec{x}] \Rightarrow C[\vec{x}]$. One obvious but inefficient way to establish this assertion is to use STE to obtain a residual and then check that $P[\vec{x}]$ implies this. But this is usually not practical. If BDDs are used, it may be too complex to compute the defining trajectory for $A[\vec{x}]$ with a symbolic simulator, for unrestricted $\vec{x}$. We might be able to get around this by using a non-canonical form such as AIGs for the formulas in our dual-rail values. But then it may anyway be intractable to check that $P[x]$ implies the residual, either by SAT or any other means.

A better way is to evaluate the defining trajectory only for variable assignments that actually do satisfy $P[\vec{x}]$. This can be done by using a *parametric representation* of $P[\vec{x}]$ to incorporate the assumptions this formula makes into the model-checking calculations of STE. Given a satisfiable $P[\vec{x}]$, we compute a vector of n propositional formulas $\vec{Q} = \text{param}(P[\vec{x}], \vec{x})$ that are substituted for the variables $\vec{x}$ in the original trajectory assertion. These formulas contain fresh Boolean variables, distinct from those in $\vec{x}$, and are constructed so that $P[\vec{Q}/\vec{x}]$ is a tautology. Moreover, for any assignment of truth-values to $\vec{x}$ that satisfies $P[\vec{x}]$, there is some valuation of all the variables in $\vec{Q}$ under which assigning the *truth-value* of each formula Q_i to the corresponding variable x_i, for $1 \leq i \leq n$, gives you this satisfying assignment to $\vec{x}$. An algorithm for computing the parametric representation of a formula and its correctness proof are given in [38].

These properties ensure that the original property $P[\vec{x}] \models A[\vec{x}] \Rightarrow C[\vec{x}]$ holds just when $\models A[\vec{Q}/\vec{x}] \Rightarrow C[\vec{Q}/\vec{x}]$ holds. They are equivalent, but the latter property has the assumption implicitly encoded into the guards of the antecedent and consequent and can be much easier to model-check. A minor complication is that any non-constant residual resulting from $A[\vec{Q}/\vec{x}] \Rightarrow C[\vec{Q}/\vec{x}]$ will be a formula over the fresh variables in $\vec{Q}$, which are an encoding artifact unlikely to be meaningful to the verification engineer. But methods exist to map such residuals back into the original variable space for counterexample analysis.

In practice, parametric representation of assumption formulas is virtually essential to the use of STE on serious examples, and is extensively used to restrict verifications to a care set and to tackle complexity by input space decomposition [3, 37, 38]. The technique is independent of the symbolic simulation algorithm in STE, does not require modifications to the circuit, and can be used to constrain both input and state values, as well as the values of internal combinational logic.

25.7 Abstraction and Symbolic Indexing

The lattice of abstract sets of states underlying STE provides an integral and flexible way to control model-checking complexity. As with all abstraction mechanisms, the strategy is to provide a means to minimize the information about circuit operation that is computationally represented when verifying a property. Crudely speaking, complexity is controlled in STE by having as many Xs as possible in place of irrelevant information about selected node values, as circuit simulation proceeds.

This mechanism of complexity control is intimately connected with the encoding, using Boolean variables, of the families of abstractions of circuit states that are computed during simulation. As already explained, propositional guards occurring in trajectory formulas introduce a layer of symbolic representation above the level of abstractions, by means of which families of abstractions are checked parametrically. The abstractions are 'indexed' by the Boolean variables present, with each valuation of the variables giving a separate abstraction case. In this setting, controlling complexity through abstraction means, in essence, controlling the complexity of the

parametric representation of node values computed during simulation. When BDDs are used for the formulas in the dual-rail encoding, this reduces the simulation-time complexity of the BDDs computed to verify a circuit property. When a non-canonical representation such as AIGs is used, this can reduce the complexity of checking the eventual problem of propositional logic that is produced by an STE model-checking run.

The mechanism of indexing by Boolean variables is pervasive in STE. Even the commonplace verification idiom of representing 'the value on a wire' symbolically by a variable is achieved through it. The most straightforward way of using STE is direct symbolic simulation with a kind of 'semantic cone-of-influence reduction' given by X-propagation. The antecedent attaches a distinct Boolean variable to each primary input that 'matters' to the property and leaves the remaining nodes X. These input values, fully determined as Boolean and named symbolically by variables, are then propagated through the circuitry to produce symbolic circuit states that are tested against the consequent. Internal node values computed during simulation may be X for some valuations of the variables, provided all the nodes inspected by the consequent have the stipulated Boolean functions of the inputs.

This simplistic approach often does not scale to complex or large circuits, and further abstraction may have to be introduced to control the complexity of model-checking. This is known as *weakening*. In essence, weakening consists in making interventions in the model-checking algorithm that result in the trajectory computed during symbolic simulation being weaker than the defining trajectory of the property being checked. If this weaker trajectory still entails the consequent, then the property still holds. If the verification fails with the weakened trajectory, then we can draw no conclusions about the original property.

More formally, recall that the fundamental theorem of trajectory evaluation says that, for a given ϕ, the assertion $\models_\phi A \Rightarrow C$ holds just when $[C]^\phi \sqsubseteq [\![A]\!]^\phi$. Now suppose we have some sequence $\sigma \sqsubseteq [\![A]\!]^\phi$ that is weaker than $[\![A]\!]^\phi$. This means that some nodes at some points in time are given values by σ that are lower in the value lattice than those given by $[\![A]\!]^\phi$. Informally, the defining trajectory has been *weakened* by throwing away knowledge about some node values. If σ still entails the defining sequence of the consequent, that is if $[C]^\phi \sqsubseteq \sigma$ still holds, then we know that $[C]^\phi \sqsubseteq [\![A]\!]^\phi$ also holds and so $\models_\phi A \Rightarrow C$ is verified.

The justification just sketched extends to the parametrically encoded families of trajectories computed by the symbolic model checking algorithm. Weakening in this case means setting the values of one or more nodes, at selected points in the simulation, to X for some or all valuations of the variables occurring in their dual-rail encoding. The dual-rail value (P_1, P_2) on a selected node at a certain point in time can simply be set to $\mathrm{X} = (\mathrm{T}, \mathrm{T})$ or, more generally, be moved arbitrarily 'closer' to X by setting it to any pair of formulas (Q_1, Q_2) for which $P_1 \supset Q_1$ and $P_2 \supset Q_2$. This makes the node have value X for more valuations of the Boolean variables at that point in time.

Forte provides fine-grained access to weakening by user-level directives that specify the nodes to weaken and—separately for each node—both the simulation times at which to weaken it and a condition on Boolean variables that stipulates

for which valuations to weaken it. Users can therefore manually weaken individual nodes at arbitrary points of time during simulation and to arbitrary degrees, with a view to reducing the BDD complexity of their dual-rail values whilst retaining just enough information about node values to satisfy the consequent. This is safe, because the theory just sketched tells us that however a node's value is weakened during a verification, if the consequent is satisfied then the trajectory assertion being checked still holds. Weakening may, however, introduce hidden vacuity (discussed in Sect. 25.4.1).

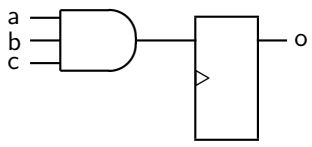
A more automated mechanism is *dynamic weakening* [65], in which a complexity threshold is set to limit the size of the BDDs representing dual-rail values. If, at any point during simulation, the size of the dual-rail value for a particular node exceeds the threshold, it is simply set to the unknown value $\mathrm{X} = (\mathrm{T}, \mathrm{T})$. This works well when the BDDs of the internal nodes along the 'relevant paths' within some complex circuitry are well-behaved, while the BDDs along other paths blow up in size. Some detailed and illuminating examples of how this mechanism can solve practical verification problems can be found in [65, pp. 1389–1390].

Intel's Core i7 processor verification effort used some more sophisticated automated weakening methods that compute 'causal fan-in information from a circuit trace to determine weakening points' [42]. The Intel engineers who conducted this impressive large-scale STE verification report that dynamic weakening together with these automated weakening techniques 'solve most circuit simulation capacity problems without need for human intervention'.

25.7.1 Symbolic Indexing

Symbolic indexing is the systematic use of weakening, controlled through the way in which antecedents are formulated, to perform partitioned abstraction that is highly effective for verifying certain regular or symmetric circuit structures. Like weakening, it is an implementation optimization that reduces model-checking complexity. But instead of only controlling BDD sizes by driving node values towards X, it also exploits symmetry to reduce the number of Boolean variables needed to verify certain circuit properties.

The idea can be illustrated by the following trivial example. Consider a unit-delay AND-gate with three inputs:



For the purpose of writing down states, we will order the nodes $\mathsf{a} < \mathsf{b} < \mathsf{c} < \mathsf{o}$. A direct STE verification that does not exploit the abstraction lattice is achieved by

checking the following trajectory assertion:

$$\mathsf{a\ is}\ v_1\ \mathsf{and\ b\ is}\ v_2\ \mathsf{and\ c\ is}\ v_3 \Rightarrow \mathsf{N}(\mathsf{o\ is}\ v_1 \wedge v_2 \wedge v_3) \tag{25.4}$$

This is just direct Boolean symbolic simulation. The antecedent attaches a distinct, unconstrained, Boolean variable to each input node, and the consequent asserts that the expected function of these variables appears on the output.

We can be more clever than this by using STE's abstraction lattice to reduce the number of Boolean variables needed to verify this gate. The key observation is that if any one input is 0, then the output will be 0 regardless of the other inputs. We can exploit this to introduce X-abstraction in the model-checking run. There are four cases to check—three in which one of the inputs is known to be 0 and the others are unknown, and one in which all three inputs are known to be 1. We can enumerate or 'index' these with two Boolean variables, say x_1 and x_2. We write the following property:

$$\begin{array}{l} \overline{x_1} \wedge \overline{x_2} \rightarrow \mathsf{a\ is\ 0\ and} \\ x_1 \wedge \overline{x_2} \rightarrow \mathsf{b\ is\ 0\ and} \\ \overline{x_1} \wedge x_2 \rightarrow \mathsf{c\ is\ 0\ and} \\ x_1 \wedge x_2 \rightarrow \mathsf{a\ is\ 1\ and\ b\ is\ 1\ and\ c\ is\ 1} \\ \quad \Rightarrow \\ \mathsf{N}(\overline{x_1} \vee \overline{x_2} \rightarrow \mathsf{o\ is\ 0\ and}\ x_1 \wedge x_2 \rightarrow \mathsf{o\ is\ 1}) \end{array} \tag{25.5}$$

Model-checking this with STE will simultaneously check all four cases, each with a different abstraction of the sets of states arising during circuit simulation. Since any property verified in STE with a node set to X also holds when the node is either 0 or 1, this verification covers all input cases and is complete.

Notice that some of the abstractions in this trivial example overlap. For example, when $x_1 = \mathrm{F}$ and $x_2 = \mathrm{F}$, the initial abstract state of the defining trajectory is 0XXX. The maximal set of Boolean states of which this is an abstraction is

$$\{0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111\}$$

When $x_1 = \mathrm{T}$ and $x_2 = \mathrm{F}$, the initial abstract state of the defining trajectory is X0XX. The maximal set of Boolean states of which this is an abstraction is

$$\{0000, 0001, 0010, 0011, 1000, 1001, 1010, 1011\}$$

The two sets have the nonempty intersection $\{0000, 0001, 0010, 0011\}$, comprising the Boolean initial circuit states covered by both abstractions. This overlapping of 'partitioned' abstraction cases is a characteristic of STE not shared by, for example, existential abstraction in CTL model checking.

Symbolic indexing finds its greatest utility in verification of regular memory structures, where it can significantly reduce the number of BDD variables required to encode data values [13, 53, 71]. Consider an $n \times m$-bit memory, with n words of memory and m bits per word. Suppose we want to verify the simple property that the read operation correctly returns the data word stored at any given address. We could write an STE property whose antecedent associates a Boolean variable $d_{i,j}$

with each bit of the initial state of the memory, for $1 \leq i \leq n$ and $1 \leq j \leq m$, and then presents a symbolic address given by a vector of Boolean variables $\vec{a}$ of length $l = \log_2 n$ on the read address input. The consequent would stipulate that the data delivered at the output is the word stored at the addressed location.

A little notation must be introduced before we can write this trajectory assertion formally. We first introduce the circuit nodes that are involved. Suppose the circuit nodes holding the individual bits of memory are systematically named '$\mathsf{mem}[i][j]$', where $1 \leq i \leq n$ identifies the word of memory and $1 \leq j \leq m$ is the bit position within the word. Let $\mathsf{mem}[i]$ be the vector of nodes $\mathsf{mem}[i][1], \ldots, \mathsf{mem}[i][m]$ for $1 \leq i \leq n$. Suppose also that the circuit nodes of the address input bits are given by the vector $\mathsf{addr} = \mathsf{addr}_1, \ldots, \mathsf{addr}_l$. Finally, suppose $\mathsf{out} = \mathsf{out}_1, \ldots, \mathsf{out}_m$ are the circuit nodes of the memory's m-bit data output.

We will need a way to associate a vector of Boolean variables, or more generally a vector of propositional expressions, with a vector of nodes. For any vector of circuit nodes $\mathsf{n} = \mathsf{n}_1, \ldots, \mathsf{n}_k$ and vector of propositional expression $\vec{P} = P_1, \ldots, P_k$, both of any length $k \geq 1$, define

$$\mathsf{n}\ \mathsf{is}\ \vec{P} \ \stackrel{\triangle}{=}\ \mathsf{n}_1\ \mathsf{is}\ P_1\ \ \mathsf{and}\ \ \cdots\ \ \mathsf{and}\ \ \mathsf{n}_k\ \mathsf{is}\ P_k$$

So '$\mathsf{n}\ \mathsf{is}\ \vec{P}$' just asserts that each circuit node in the vector n has the value given by the respective Boolean expression in the vector $\vec{P}$.

As mentioned, the antecedent of our trajectory assertion will associate Boolean variables $d_{i,j}$ with the individual bits of memory state, where $1 \leq i \leq n$ is the memory location and $1 \leq j \leq m$ is the bit position within that location. To express this, we will write $\vec{d}_i = d_{i,1}, \ldots, d_{i,m}$ for the vector of m Boolean variables associated with the bits of the word stored at memory location i.

To formulate the consequent, we first need a way to state, in propositional logic, that the address input $\vec{a}$ has a specific value in interval $[1,n]$. It is routine to encode this by propositional assertions '$\vec{a} \sim i$', where $1 \leq i \leq n$, that hold exactly when the address bits $\vec{a}$ form the unsigned Binary number corresponding to the integer $i-1$. We can then define propositional expressions $d_{\vec{a},j}$, for $1 \leq j \leq m$ that give the jth bit of the data word at the memory location addressed by $\vec{a}$. Define

$$d_{\vec{a},j} \ \stackrel{\triangle}{=}\ (\vec{a} \sim 1 \wedge d_{1,j}) \vee \cdots \vee (\vec{a} \sim n \wedge d_{n,j}).$$

We are now in a position to express the required trajectory assertion formally:

$$\begin{array}{l} \mathsf{addr}\ \mathsf{is}\ \vec{a}\ \ \mathsf{and} \\ \mathsf{mem}_1\ \mathsf{is}\ \vec{d}_1\ \mathsf{and}\ \ \cdots\ \ \mathsf{and}\ \mathsf{mem}_n\ \mathsf{is}\ \vec{d}_n \end{array} \quad \Rightarrow \quad \mathsf{out}\ \mathsf{is}\ d_{\vec{a},1}, \ldots, d_{\vec{a},m}$$

The antecedent simply associates a Boolean variable with each bit of memory state and introduces a vector of Boolean variables to name the bits of the read address. The consequent says that each bit of the data output will be the bit at the corresponding position within the word stored at the given address.

Distinguishing each memory location in this direct verification requires $n \times m$ unique Boolean variables. There are a further $l = \log_2 n$ Boolean variables for the address input. For even a small memory, the number of variables used and the complexity of the state obtained during simulation are too large for symbolic verification. But with symbolic indexing we can get away with only m variables, $\vec{d} = d_1, \ldots, d_m$ say, to represent the data in *only the addressed* memory location. We write the trajectory assertion as follows:

$$\begin{array}{l} \mathsf{addr\ is}\ \vec{a}\ \ \mathsf{and} \\ (\vec{a} \sim 1 \rightarrow \mathsf{mem}_1\ \mathsf{is}\ \vec{d})\ \mathsf{and}\ \cdots\ \mathsf{and}\ (\vec{a} \sim n \rightarrow \mathsf{mem}_1\ \mathsf{is}\ \vec{d}) \end{array} \quad \Rightarrow \quad \mathsf{out\ is}\ \vec{d}$$

Here, the antecedent assumes that the memory node storing the jth bit of the ith location has the Boolean value d_j under a guard $\vec{a} \sim i$ stating that the ith row is in fact addressed. For example, when the address bits select location 1, the memory node $\mathsf{mem}[1][j]$ is set to d_j, for $1 \leq j \leq m$. But when the address is not 1, these nodes are set to the unknown value X. (It is important to recall that all cases are simulated simultaneously in STE.) Now, with this indexed arrangement, we can expect the jth output node always to have the same Boolean value d_j whatever the value of the address bits, which is what the consequent stipulates. In this verification by symbolic indexing, only $m + l$ variables are required—a very significant reduction. Moreover, in a BDD-based verification, the BDDs representing the bits on each column in the memory array will have a significant amount of sharing, since each of these bits is essentially driven by the same Boolean variable but under different guard conditions. The result is an extremely efficient memory verification computation.

Symbolic indexing is highly effective and widely used in memory verification, and similar circuits such as CAMs, but has seen relatively limited adoption for other circuit structures. This is because users have to create the right indexed family of abstractions by manually encoding them into the antecedent. To make symbolic indexing easier, Melham and Jones [48] describe an algorithm for computing a trajectory assertion that encodes a user-given symbolic indexing scheme from a trajectory assertion that specifies a direct symbolic simulation of the circuit. The transformation is sound, in the sense that if the resulting trajectory assertion holds then so does the original one. This method, however, still requires the user to specify the indexing scheme, albeit with much less effort than manually writing it into the antecedent.

An algorithm that leverages this transformation to automatically abstract properties by symbolic indexing is presented in [5]. This takes as input a specification for the circuit to be verified, in the form of a Boolean expression that states the required I/O function for the circuit. By analyzing the information requirements for intermediate computations in the specification, the algorithm computes a candidate scheme for symbolic indexing that can then be applied to the verification property using the Melham and Jones transformation. Experimental results show that this approach not only simplifies memory verification, but also enables symbolic indexing of certain other designs fully automatically.

Another technique for memory verification that employs a very different form of symbolic indexing was introduced by a company called InnoLogic Systems, which

was later acquired by Synopsys [58]. The technique, described in [79], uses symbolic Boolean variables to encode regular circuit structures, so that a single simulation 'node' records the state values for multiple identical circuit nodes. This enables the verification of very large regular memory arrays, since this encoding overcomes the need to have state information proportional to the number of circuit nodes.

25.8 Compositional Reasoning

The practical application of symbolic trajectory evaluation—indeed of any model-checking method—to complex, industrial-scale circuit designs inevitably faces the serious challenge of fundamental capacity limitations. STE's native abstraction mechanism helps with scalability, and in particular can make the method scale well for circuits whose semantics admits of a good symbolic indexing scheme. But for many industrial-scale design verifications, abstraction must be complemented by some form of decomposition into sub-problems that STE model-checking can handle. To fit into the capacity of the model-checker, a high-level correctness property may have to be broken down into many individual trajectory assertions, which combine in potentially complex ways to establish the overall correctness result [52, 65].

To provide a theoretical foundation for problem decomposition, STE can be equipped with a system of formal inference rules for proving trajectory assertions from an axiomatic base of assertions generated by STE model checking [31, 80]. In Sect. 25.8.1, we give a brief overview of one formulation of such a system. We then sketch, in Sect. 25.8.2, an alternative approach to compositional reasoning, called 'Relational STE' [52]. This bypasses the language of trajectory assertions entirely, raising verification properties to a purely logical level suitable for compositional reasoning in ordinary predicate logic.

The use of mechanized, deductive theorem proving—as exemplified by the HOL system [27]—has often been proposed to support compositional reasoning using systems of STE inference rules. The combination of algorithmic STE model checking and deductive proof was pioneered in the early 1990s in an academic predecessor of Intel's Forte system called Voss [62]. This was followed by numerous experiments in designing and using systems that linked STE and theorem proving of various kinds [2, 4, 33, 40, 65, 66], culminating in a mature integration within Forte of a comparatively full-featured theorem prover, called 'Goaled'. This provides an integrated combination of STE and theorem proving that is seeing increasing use in production verification projects at Intel [52].

25.8.1 The STE Deductive System

In this section, we briefly sketch a system of STE inference rules originally presented by Hazelhurst and Seger in [31]. Another formulation can be found in Hazel-

hurst's PhD dissertation [30] and is also presented and illustrated by examples in the tutorial [32]. The main use of these deductive systems is to combine individual STE model-checking results together to derive correctness properties that are infeasible to check directly. But the rules can also be used to transform correctness assertions to increase STE model-checking efficiency [2].

Rules of Consequence. These rules are analogous to the classical Hoare logic rules for pre-condition strengthening and post-condition weakening, and are used in proofs for a similar purpose: aligning antecedents and consequents to enable further deductions, primarily transitivity. They are defined semantically, via the ordering on defining sequences, rather than in terms of syntactic implication:

Antecedent Strengthening. For any trajectory formulas A and C, if $\models A \Rightarrow C$ then for any trajectory formula A' for which $[A]^\phi \sqsubseteq [A']^\phi$ for all valuations ϕ, we have $\models A' \Rightarrow C$.

Consequent Weakening. For any trajectory formulas A and C, if $\models A \Rightarrow C$ then for any trajectory formula C' for which $[C']^\phi \sqsubseteq [C]^\phi$ for all valuations ϕ, we have $\models A \Rightarrow C'$.

The Antecedent Strengthening rule says that the antecedent of any proved trajectory assertion can be replaced by one that adds further information about node values, i.e. that has 0 or 1 in place of some Xs, under some valuations of the Boolean variables in the guards. In essence, this rule encapsulates the fundamental abstraction mechanism of STE: complexity is controlled by abstracting away from irrelevant information about node values, and running the STE model checking algorithm with the weakest possible antecedent that still establishes the consequent. Similarly, Consequent Weakening says that the consequent of a proved trajectory assertion can be replaced by one with less information about node values—you can discard established information about node values.

Logical and Structural Rules. The first of these is a simple axiom,

Reflexivity. $\models A \Rightarrow A$ holds for any trajectory formula A,

that provides a syntactic starting point for deductive proofs. Of course the other way to begin a deduction is to establish one or more trajectory assertions semantically, by STE model checking.

The next two inference rules allow compositional reasoning about complex circuit behaviours, in which 'larger' circuit properties are built up from 'smaller' ones:

Conjunction. For any trajectory formulas A_1, A_2, C_1, and C_2, if $\models A_1 \Rightarrow C_1$ and $\models A_2 \Rightarrow C_2$, then $\models A_1 \text{ and } A_2 \Rightarrow C_1 \text{ and } C_2$.

Transitivity. For any trajectory formulas A, B, and C, if $\models A \Rightarrow B$ and $\models B \Rightarrow C$, then $\models A \Rightarrow C$.

Conjunction allows circuit properties proved separately, typically because of model checking capacity limits—or because different parts of a large circuit must be simulated under different BDD orderings—to be combined into a single property. If the assumptions expressed by both antecedents hold, then the conjunction of the consequents is established. Transitivity allows one to break a simulation of circuit behaviour over a long period of time into a succession of smaller intervals. Starting from the antecedent A, first establish some relationship B among the values on some intermediate circuit nodes by proving $\models A \Rightarrow B$. Then show that simulation beginning with the assumption that B holds establishes the final consequent C.

Time Shift. This rule allows the baseline time at which simulation of the circuit begins in its initial state to be shifted forward. This is used, for example, to align time points to allow transitivity to be applied. It also allows a single, general assertion to be instantiated for use in many contexts, differing only in the specific finite interval of time over which the instantiated property needs to specify behaviour. The inference rule is the following:

Time Shift. For any trajectory formulas A and C, if $\models A \Rightarrow C$ then $\models \mathsf{N}A \Rightarrow \mathsf{N}C$.

Time Shift holds because, in STE, the initial state at which simulation begins is arbitrary. The only constraints on it are those explicitly imposed by the antecedent, as discussed in Sect. 25.5.

Substitution. The final rule is the only one (in this formulation) that has specifically to do with Boolean variables and guards. In simplified form, the rule is

Substitution. For any trajectory formulas A and C, if $\models A \Rightarrow C$, then $\models A[\vec{P}/\vec{x}] \Rightarrow C[\vec{P}/\vec{x}]$ for any substitution of formulas $\vec{P}$ for Boolean variables $\vec{x}$.

A somewhat more flexible form of the rule, which deals with some essentially syntactic complexities, is presented in [31]. A primary use of Substitution is, again, to align antecedents and consequents so transitivity applies. Suppose, for example, we have proved some unit-delay input-output relationship $\models \mathsf{a}\ \mathsf{is}\ x \Rightarrow \mathsf{N}(\mathsf{b}\ \mathsf{is}\ E_1)$. Now suppose we separately simulate the next stage in the circuit's sequential behaviour, setting node b to a fresh variable y and proving $\models \mathsf{b}\ \mathsf{is}\ y \Rightarrow \mathsf{N}(\mathsf{c}\ \mathsf{is}\ E_2)$. Using time-shifting, we obtain $\models \mathsf{N}(\mathsf{b}\ \mathsf{is}\ y) \Rightarrow \mathsf{N}(\mathsf{N}(\mathsf{c}\ \mathsf{is}\ E_2))$. We can then use Substitution to replace the input variable y for the second stage by the output expression E_1 actually computed in the first stage, to obtain $\models \mathsf{N}(\mathsf{b}\ \mathsf{is}\ E_1) \Rightarrow \mathsf{N}(\mathsf{N}(\mathsf{c}\ \mathsf{is}\ E_2[E_1/y]))$. The two stages can then be composed using Transitivity.

As already mentioned, several software tools that combine STE and deductive theorem proving have been designed to support compositional reasoning using this and other systems of specialized STE inference rules. These commonly embed reasoning about STE properties within a more general, higher-order logic: a formalized syntax is introduced that constitutes a 'deep embedding' of trajectory formulas and assertions, and then the inference rules are added to give an axiomatic theory of this

embedded 'STE logic'. The connection to model checking is achieved by an axiom-scheme that admits any trajectory assertion that has been (externally) checked by the STE model checking algorithm.

This approach yields a two-level integration, in which the deductive system for inferring circuit properties is largely separate from the general higher-order logic of the theorem prover. Some bridges between the two levels can, however, be achieved by adding certain quantifier rules and axioms about parametric encoding of assumptions. See Sect. VII of [65] for details.

25.8.2 Relational STE

The primitive language of STE trajectory assertions, introduced in Sect. 25.4.1, in essence requires a circuit specification to be functional. Given some inputs, the specification stipulates the values that the outputs shall have, potentially under some constraints that the inputs must satisfy. But many informal circuit specifications seen in practice do not fall into this category—the simplest example being 'nodes a and b are mutually exclusive'. Such *relational* specifications become especially important at higher levels of abstraction, where it may be most appropriate for specifications to be partial [52]. For example, a natural specification of a processor's micro-operation scheduler might say 'a micro-operation with ready sources will be scheduled for execution', while intentionally leaving open the selection between different ready micro-operations. Even at lower abstraction levels, the most natural form of specification may not be functional.

Relational STE is an approach to compositional reasoning about circuit properties that retains STE's underlying symbolic simulation engine but lifts the properties to the level of ordinary predicate logic [52]. This allows much more general properties to be expressed—in particular, properties can be relations, rather than only I/O functions. Perhaps equally important, it also avoids the rather intricate, low-level process of reasoning with specialized STE inference rules illustrated in Sect. 25.8.1. Developed as an extension to Intel's Forte system, Relational STE has been used very effectively for difficult, industrial-scale verifications of floating-point dividers [43] and control-dominated microarchitectural algorithms, such as bus recycle logic and register renaming [41]. These are all circuits for which the natural form of specification is relational.

We now briefly sketch Relational STE, first establishing some basic terminology. Given a circuit c with nodes $\mathscr{N}$, an *execution* is a function $e \in (\mathscr{N} \times \mathbb{N}) \to \mathbb{B}$ that assigns a Boolean value to each circuit node at each point in time. (So an execution is just an STE sequence that is Boolean-valued; see Sect. 25.3.3.) The *behaviour* of a circuit is a set of all its possible executions. We write

$$\|c\| \subseteq (\mathscr{N} \times \mathbb{N}) \to \mathbb{B}$$

for the behaviour of a circuit c. This is simply the classical 'relational' approach to representing circuit behaviour mathematically that is well-known from hardware modelling in higher-order logic [17, 47].

Relational STE uses the symbolic simulation algorithm at the heart of STE to check that the behaviours of a circuit, in the sense just defined, satisfy specifications formulated as *constraints*. In essence, a constraint

$$p \subseteq (\mathscr{N} \times \mathbb{N}) \to \mathbb{B}$$

is a predicate on circuit executions that requires a certain relationship to hold among the values that appear on a stipulated set of circuit nodes at some individually specified points of time. The *signature* of a constraint is a finite subset of $\mathscr{N} \times \mathbb{N}$ that defines what these nodes and times are.

Constraints define relationships that are expected to hold among the values on some circuit nodes at certain times. Consider, for example, the informal specification mentioned earlier: 'circuit nodes a and b are mutually exclusive at time point 2'. This can be expressed by the constraint

$$\{e \in (\mathscr{N} \times \mathbb{N}) \to \mathbb{B} \mid \overline{e(\mathsf{a},2) \wedge e(\mathsf{b},2)}\}$$

with signature $\{(\mathsf{a},2),(\mathsf{b},2)\} \subseteq \mathscr{N} \times \mathbb{N}$. This represents a predicate that holds of just those circuit executions in which nodes a and b are mutually exclusive at time 2. Note that this form of specification is expressed—in essence at least—through ordinary (higher-order) propositional logic. For concise presentation, we have explained constraints in set theoretic notation. But it's not hard to see this as a higher-order predicate on functions:

$$\text{mutex } e \stackrel{\triangle}{=} \overline{e(\mathsf{a},2) \wedge e(\mathsf{b},2)}$$

A theorem of higher-order logic that says all the executions of a circuit c satisfy this predicate would then look something like $\vdash \forall e : (\mathscr{N} \times \mathbb{N}) \to \mathbb{B} \,.\, e \in \|c\| \supset \text{mutex } e$.

In practice, circuit correctness properties are formulated in Relational STE as a pair of constraints: an *input constraint*, P_i, and an *output constraint*, P_o. Given such a pair, the underlying symbolic simulation algorithm of STE is used to check that any circuit execution that satisfies the input constraint will (under simulation) also satisfy the output constraint. Formally, Relational STE proves correctness statements of the form $\vdash \forall e : (\mathscr{N} \times \mathbb{N}) \to \mathbb{B} \,.\, e \in \|c\| \supset (\mathrm{P}_i\, e \supset \mathrm{P}_o\, e)$.

The algorithm for verifying these relational correctness properties is simple, at least conceptually: it is just Boolean symbolic simulation. First construct a classical STE antecedent that attaches a unique, unconstrained Boolean variable to every circuit node. Then run the STE simulator—for as many steps as needed, according to the signatures of the input and output constraints—and record all the symbolic node values that arise at each step of the simulation. This produces a single, *symbolic* execution, $\hat{e}$ say, that encompasses all the behaviours of the circuit. The implication $\mathrm{P}_i\, \hat{e} \supset \mathrm{P}_o\, \hat{e}$ can then be evaluated to get the result using BDDs or a SAT solver.

Of course this simplistic algorithm is vastly too complex, computationally, to be practical for industrial-scale problems. The implementation of Relational STE in Intel's Forte system employs a sophisticated array of optimizations that exploit the power of native STE. For example, not all nodes are set to variables at the start of simulation, only the ones needed to evaluate the input and output constraints. (Or that have to be Boolean to avoid weak disagreements; see Sect. 25.6.) The remaining inputs have value X at the start of the simulation, raising the general level of abstraction and lowering complexity. In addition, many inputs are typically set to constants, for example clock patterns and certain testability signals. More importantly, some elements of the input constraint will be injected into the simulation using parametric representation, as discussed in Sect. 25.4.2. Dynamic weakening and some other abstraction mechanisms are also used. Exploitation of the full power of symbolic indexing, however, remains for future development of the framework [52].

This relational form of symbolic trajectory evaluation preserves the power of the underlying STE algorithm while enabling much richer specifications: ones that stipulate relations among nodes values, rather than just functions. Relational STE also eliminates the need to use specialized STE inference rules and apparatus for temporal reasoning; the relational formulation makes it 'just' higher-order logic. Since its introduction, Relational STE has been the workhorse of data-path formal verification at Intel; many thousands of individual operations have been verified in several microprocessor families and over the course of several generations [52].

25.9 GSTE and other Extensions

Relational STE essentially discards the fixed temporal logic of STE trajectory logic in favour of much less restricted assertions about behaviour under simulation, expressed in conventional predicate logic. This represents a step away from the classical idea of 'model checking' temporal formulas. Almost since the inception of STE, however, variants and extensions of STE have been formulated that remain in the realm of temporal model checking, but have more expressive specification languages than the simple language of trajectory formulas and trajectory assertions introduced in Sect. 25.4.

In their cornerstone article on STE [64], Seger and Bryant formulate a version with higher expressive power at the level of trajectory *assertions*. In addition to the basic form of trajectory assertion, $A \Rightarrow C$, they allow sequences $A \Rightarrow C\,;\, G$ and iterations $(A \Rightarrow C)^\star;\, G$, where G is another trajectory assertion. Sequences allow circuit behaviours to be specified as the concatenation of a temporal succession of sub-behaviours. Roughly speaking, an iteration $(A \Rightarrow C)^\star;\, G$ specifies that any trajectory of the circuit must satisfy C for as long as it satisfies A, after which it will satisfy G. Neither form of correctness assertion is reported to have found widespread use in industrial practice.

In their comprehensive STE tutorial [32], Hazelhurst and Seger give a version of STE in which the property language, called TL, is a four-valued linear-time temporal

logic that includes negation and a strong Until operator. In contrast to the trajectory formulas of Sect. 25.4, the formulas of TL have four possible *truth values*, true (**t**), false (**f**), unknown ($\bot$), and inconsistent ($\top$), ordered in a lattice in the obvious way. It is important to distinguish between this four-valued lattice of *truth values* of TL formulas and the lattice of *state values*, ordered by information content, introduced in this chapter in Sect. 25.3.1. The former represents the *degree of knowledge* we have of the truth or falsity of propositions in the property language; the latter captures the *amount of information* there is about the values on circuit nodes. Hazelhurst and Seger give an interesting justification for maintaining this distinction [32, p.19], a key point of which is that it allows a formulation of STE with negation.

The theory of STE in this extended form is a little more involved than the simpler version presented in this chapter. For example, there is no longer a single 'defining' sequence and trajectory for a formula—instead TL formulas have defining *sets* of minimal sequences and trajectories. An analogue of the Fundamental Theorem of Trajectory Evaluation (see Sect. 25.5) can be obtained in this system, but exploiting this to actually check properties in an algorithmic way is rather more involved than the method sketched in Sect. 25.6. In practice, certain restrictions must be placed on the formulas checked, and some approximations are involved. Full details of the theory and model-checking algorithms—as well as a theory of compositional reasoning—can be found in Hazelhurst's Ph.D. dissertation [30].

This chapter has focussed on *bit-level* hardware models and their verification in symbolic trajectory evaluation—a level of hardware modelling at which STE has been spectacularly successful for industrial data-path verification. Right from the start, however, the theory of STE was framed in the much more general setting of an arbitrary lattice of state abstractions [64], and was not restricted to the bit-level abstraction of Boolean states introduced in Sect. 25.3.2. The idea was that one could have STE-style algorithms that analyse systems at a higher (or at least different) level of *data abstraction* [47], provided a good representation for the indexed families of abstract states could be devised. The arrival of highly efficient SMT solvers has paved the way for this ambition to be realised in the form of *word-level STE*. This is a verification method based on symbolic simulation and a lattice of abstract states in which the underlying notion of concrete data is a group of binary digits, rather than individual bits. Work is underway at Intel to create such word-level STE verification tools [63].

25.9.1 Generalized Symbolic Trajectory Evaluation

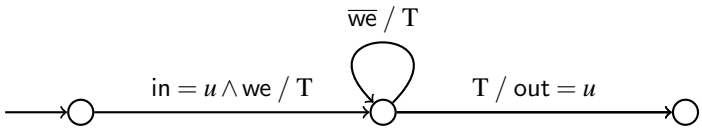
A more radical departure from classical STE is Generalized Symbolic Trajectory Evaluation, GSTE. This aims to preserve the power of abstraction and efficiency benefits of STE while making the property language much more expressive, using some of the fundamental techniques of classical symbolic model checking.

GSTE builds on several prior efforts to increase the expressive power of STE, including the introduction of iterations in [64]. Beatty first generalized specifications

to arbitrary labelled transition graphs [7], and a model checking algorithm for these specifications was proposed by Nelson and Jain [49]. Jain proposed a semantics that existentially quantifies over paths in a transition graph and gave an incomplete model-checking algorithm [36]. The next step was the insightful development by Chou of a more natural semantics that universally quantifies over paths, together with a sound and complete model-checking algorithm [19]. Yang and Seger then introduced GSTE, using a graphical specification notation, and adding backwards reasoning and other algorithmic developments [77, 78]. GSTE has also been further extended to introduce a variant called *concurrent* (or *compositional*) GSTE [73, 76]. This allow different parts of circuits to be analyzed by independent simulations and the results combined.

GSTE overcomes the two main limitations of classical STE. It can express properties over *unbounded* periods of time, and it allows reasoning by propagation of information *backward* in time. The resulting extension is significant—algorithms for GSTE can verify all ω-regular properties [78]. Specifications are presented in a notation called *assertion graphs*. These are directed graphs with an initial vertex, where each edge is labelled by *antecedent* and *consequent* conditions on circuit nodes. The vertexes represent sets of circuit states and the edges discrete transitions between them. As with conventional STE, the antecedents drive simulation with a constrained input stimulus and the consequents stipulate the expected circuit response. Roughly speaking, the property specified by an assertion graph is the following. The graph defines a set of finite paths, each starting at the initial vertex. For each such path, all finite sequences of circuit states of the same length that satisfy all the antecedents along the path must also satisfy all the consequents along the path.

A simple example, taken from [69], is the memory cell specification below:



The edges are labelled with pairs of the form *antecedent* / *consequent*. The property specified is that whenever the write enable node we is high and the input node has the value named symbolically by u, the output node out should subsequently present the value u for as long as no further writes are enabled. Much more complex examples can be found in the literature. The verification of FIFO circuits is explored in [75], and a FIFO case study is again used to illustrate GSTE in [74]. The specification and verification of a complex memory unit is explained in [78]. Concurrent GSTE is applied to the verification of Intel Pentium 4 scheduler circuity in [73, 76].

The GSTE model-checking algorithm presented in [78] records a set of states of the circuit model for each assertion graph edge. This contains all the states that are reachable along some path from the initial vertex via a trace of circuit execution that satisfies all the antecedents along the path. The algorithm is similar to reachability analysis, computing the fixed point of taking the post-images of each transition. This and other connections between GSTE and conventional symbolic model checking are discussed in [60]. Overlaid onto this basic algorithmic framework are a number

of technical devices for controlling and localizing abstraction levels across an assertion graph. These include a mechanism for limiting the scope of symbolic variables to a specific edge [75] and for manually controlling certain other points at which variables are quantified—so called *knots* [50]. Another variable handling mechanism, called *precise nodes* [75], is used to maintain node value inter-dependencies across temporally overlapping scopes.

As discussed in Sect. 25.4.1, an essential limitation of conventional formulations of STE is that they reason by forwards simulation only. They therefore cannot verify properties that require the propagation of information backwards along state transitions. GSTE aims to overcome this by adding an initial phase to the model-checking algorithm; a pre-image fixed point calculation is done that strengthens earlier antecedent constraints in the assertion graph by propagating later antecedent constraints backwards. See [75, 78] for more details of this algorithm.

Although Generalized Symbolic Trajectory Evaluation has seen some success at Intel Corporation, it has not—at the time of writing—yet been established in widespread use. Moreover, as some of the practical complexities just hinted at suggest, GSTE may not yet be mature and may see further development and modification. For example, Smith's PhD dissertation recasts GSTE into a system comprising *generalized trajectory logic*, a low-level temporal logic for reasoning about symbolic ternary simulations, and *assertion programs*, an executable high-level specification language [69]. The resulting system is, it is claimed, cleaner and more amenable to formal reasoning. We therefore do not give full technical details of GSTE in this chapter, but refer the reader to the literature cited in this section. Good starting points for learning about GSTE in depth are the background section of Smith's PhD dissertation [69, Sect. 2.5] and the semantic account by Roorda and Claessen [21].

25.10 Summary and Prospects

STE and its descendent GSTE provide a uniquely effective approach to formal verification of difficult, industrial scale circuit designs—especially data-paths, but some control-dominated designs too. Their effectiveness comes from a combination of symbolic simulation with a two-level system of abstraction, whereby (overlapping) families of abstractions can be represented compactly and checked simultaneously. Intel Corporation has been a prominent user, and developer, of this model-checking technology. At the time of writing, Intel's deployment of STE through its Forte environment was one of the most substantial and sustained formal (property) verification efforts anywhere in industry. But STE-based formal verification has also seen significant use at Motorola and IBM.

More recently, an in-house verification framework has been developed at Centaur Technology for processor verification that has many parallels with Intel's Forte environment, as well as some significant differences [35, 67]. This, too, is based on symbolic circuit simulation, provides abstraction, and has a logic for compositional

reasoning. A notable feature of the Centaur framework is that it is built on top of publicly-available software tools: the well-established ACL2 [44] theorem prover and special-purpose tools such as the ZZ framework [25] and ABC [9].

The successful industrial deployment of two major verification frameworks based on symbolic simulation—Forte at Intel and the ACL2-based tools at Centaur Technology—suggest that this idea has come of age industrially, at least for processor verification. Moreover the parallels between the two systems, each quite different from the other in numerous matters of detail, strengthens the conclusion that this *kind* of approach represents a general solution in this important domain. To date, there are no commercial STE tools, though the technology is well documented and seems ripe for more widespread take-up.

GSTE is not as well established as STE, but seems to be a very promising idea that merits much further development and experimentation. Perhaps the best way to think of GSTE, with its assertion graphs or (in Smith's formulation) assertion programs, is as a framework for articulating 'reference' hardware algorithms at a flexible, intermediate layer of abstraction above the circuit level.

There is, of course, a very sizable literature on partitioning and abstraction in both hardware and software verification. A few connections have been made in the literature between the use of these ideas in other settings and the specific mechanisms provided in STE and GSTE. For example, a combination of partitioned abstraction, similar to that provided in GSTE and STE, with predicate abstraction and counterexample-guided abstraction refinement in the context of symbolic model checking, is explored in [61]. A fruitful research direction would be to look for other ways in which underlying ideas of STE and GSTE, which have made them so successful in processor verification, might be adapted to other contexts, especially software verification.

Acknowledgements Carl Seger and Randy Bryant are the originators of STE, and GSTE is due to Jin Yang and Carl Seger. I am grateful for many illuminating discussions of STE with Carl, and extremely grateful for an extended and close collaboration with Carl and many other Intel scientists and engineers who have contributed to STE and its embodiment in the Forte system. Ed Smith's DPhil dissertation provided a fresh perspective on GSTE. John Harrison, my students at Oxford, and the anonymous referees all provided insightful comments that improved the presentation.

References

1. M. D. Aagaard, R. B. Jones, T. F. Melham, J. W. O'Leary, and C.-J. H. Seger. A methodology for large-scale hardware verification. In W. A. Hunt, Jr. and S. D. Johnson, editors, *Formal Methods in Computer-Aided Design: Third International Conference, FMCAD 2000: Austin, TX, USA, November 1–3, 2000: Proceedings*, volume 1954 of *Lecture Notes in Computer Science*, pages 263–282. Springer-Verlag, 2000.
2. M. D. Aagaard, R. B. Jones, and C.-J. H. Seger. Combining theorem proving and trajectory evaluation in an industrial environment. In *ACM/IEEE Design Automation Conference*, pages 538–541, 1998.

3. M. D. Aagaard, R. B. Jones, and C.-J. H. Seger. Formal verification using parametric representations of Boolean constraints. In *ACM/IEEE Design Automation Conference*, pages 402–407. ACM Press, June 1999.
4. M. D. Aagaard, R. B. Jones, and C.-J. H. Seger. Lifted-FL: A pragmatic implementation of combined model checking and theorem proving. In Y. Bertot, G. Dowek, A. Hirschowitz, C. Paulin, and L. Théry, editors, *Theorem Proving in Higher Order Logics*, volume 1690 of *Lecture Notes in Computer Science*, pages 323–340. Springer-Verlag, 1999.
5. S. Adams, M. Björk, T. Melham, and C.-J. Seger. Automatic abstraction in symbolic trajectory evaluation. In Jason Baumgartner and Mary Sheeran, editors, *Formal Methods in Computer Aided Design: FMCAD 2007: November 11–14 2007, Austin, Texas, USA*, pages 127–135. IEEE Computer Society, 2007.
6. T. Ball, A. Podelski, and S. K. Rajamani. Boolean and Cartesian abstraction for model checking C programs. *International Journal on Software Tools for Technology Transfer (STTT)*, 5:49–58, 2003.
7. D. L. Beatty and R. E. Bryant. Formally verifying a microprocessor using a simulation methodology. In *Proceedings of the 31st annual Design Automation Conference*, pages 596–602. ACM, 1994.
8. I. Beer, S. Ben-David, C. Eisner, and Y. Rodeh. Efficient detection of vacuity in ACTL formulas. *Formal Methods in System Design*, 18(2):141–162, March 2001.
9. Berkeley Logic Synthesis and Verification Group. ABC: A system for sequential synthesis and verification. http://www.eecs.berkeley.edu/ alanmi/abc/.
10. J. Bhadra, A. Martin, J. Abraham, and M. Abadir. A language formalism for verification of PowerPC custom memories using compositions of abstract specifications. In *High-Level Design Validation and Test Workshop, 2001: Proceedings*, pages 134–141. IEEE Computer Society, 2001.
11. J. Bhadra, A. K. Martin, J. A. Abraham, and M. S. Abadir. Using abstract specifications to verify PowerPC custom memories by symbolic trajectory evaluation. In *Correct Hardware Design and Verification Melthods: CHARME 2001*, volume 1690 of *Lecture Notes in Computer Science*, pages 386–402. Springer-Verlag, 2001.
12. R. Bird. *Introduction to Functional Programming using Haskell*. Prentice Hall, second edition, 1998.
13. R. E. Bryant. Formal verification of memory circuits by switch-level simulation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 10(1):94–102, January 1991.
14. R. E. Bryant. A methodology for hardware verification based on logic simulation. *Journal of the ACM*, 38(2):299–328, April 1991.
15. R. E. Bryant, D. Beatty, K. Brace, K. Cho, and T. Sheffler. COSMOS: a compiled simulator for MOS circuits. In *Proceedings of the 24th ACM/IEEE Design Automation Conference*, DAC '87, pages 9–16. ACM, 1987.
16. R. E. Bryant, D. E. Beatty, and C.-J. H. Seger. Formal hardware verification by symbolic ternary trajectory evaluation. In *ACM/IEEE Design Automation Conference*, pages 297–402, 1991.
17. Albert Camilleri, Mike Gordon, and Tom Melham. Hardware verification using higher-order logic. In Dominique Borrione, editor, *From HDL Descriptions to Guaranteed Correct Circuit Designs: Proceedings of the IFIP WG 10.2 Working Conference on From HDL Descriptions to Guaranteed Correct Circuit Designs, Grenoble, France, 9–11 September, 1986*, pages 43–67. North-Holland, 1987.
18. H. Chockler, O. Grumberg, and A. Yadgar. Efficient automatic STE refinement using responsibility. In R. Ramakrishnan and J. Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems: 14th International Conference, TACAS 2008*, volume 4963 of *Lecture Notes in Computer Science*, pages 233–248. Springer-Verlag, 2008.
19. C.-T. Chou. The mathematical foundation of symbolic trajectory evaluation. In N. Halbwachs and D. Peled, editors, *Computer Aided Verification: 11th International Conference, Trento, Italy, July 6–10 1999: Proceedings*, volume 1633 of *Lecture Notes in Computer Science*. Springer-Verlag, 1999.

20. K. Claessen and J.-W. Roorda. An introduction to symbolic trajectory evaluation. In *Formal Methods for Hardware Verification, 6th International School on Formal Methods for the Design of Computer, Communication, and Software Systems, SFM 2006, Bertinoro, Italy, May 22-27, 2006, Advanced Lectures*, volume 3965 of *Lecture Notes in Computer Science*, pages 56–77. Springer, 2006.
21. K. Claessen and J.-W. Roorda. A faithful semantics for generalised symbolic trajectory evaluation. *Logical Methods in Computer Science*, 5(2):1–32, 2009.
22. E. M. Clarke, O. Grumberg, and D. E. Long. Model checking and abstraction. In *Proceedings of the 19th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 343–354. ACM, 1992.
23. E. M. Clarke, O. Grumberg, and D. Peled. *Model Checking*. MIT Press, 1999.
24. B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 1990.
25. Niklas Een. ABC/ZZ. https://bitbucket.org/niklaseen/abc-zz.
26. A. Flaisher, A. Gluska, and E. Singerman. Case study: Integrating FV and DV in the verification of the Intel Core 2 Duo microprocessor. In J. Baumgartner and M. Sheeran, editors, *Formal Methods in Computer Aided Design: FMCAD 2007*, pages 192–195. IEEE Computer Society, 2007.
27. M. J. C. Gordon and T. F. Melham, editors. *Introduction to HOL: A theorem proving environment for higher order logic*. Cambridge University Press, 1993.
28. O. Grumberg. Abstraction and refinement in model checking. In Frank de Boer, Marcello Bonsangue, Susanne Graf, and Willem-Paul de Roever, editors, *Formal Methods for Components and Objects*, volume 4111 of *Lecture Notes in Computer Science*, pages 219–242. Springer Verlag, 2006.
29. P. R. Halmos. *Naive Set Theory*. Springer-Verlag, 1987.
30. S. Hazelhurst. *Compositional Model Checking of Partially Ordered State Spaces*. Technical report 96-02, Department of Computer Science, University of British Columbia, 1996.
31. S. Hazelhurst and C.-J. H. Seger. A simple theorem prover based on symbolic trajectory evaluation and BDDs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits*, 14(4):413–422, April 1995.
32. S. Hazelhurst and C.-J. H. Seger. Symbolic trajectory evaluation. In T. Kropf, editor, *Formal Hardware Verification*, chapter 1, pages 3–78. Springer-Verlag, 1997.
33. S. Hazelhurst and C.-J.H. Seger. A simple theorem prover based on symbolic trajectory evaluation and BDD's. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 14(4):413–422, 1995.
34. L. Hellerman. A catalog of three-variable or-invert and and-invert logical circuits. *IEEE Transactions on Electronic Computers*, EC-12(3):198–223, June 1963.
35. W. A. Hunt, Jr., S. Swords, J. Davis, and A. Slobodova. *Use of Formal Verification at Centaur Technology*, pages 65–88. Springer-Verlag, 2010.
36. A. Jain. *Formal Hardware Verification by Symbolic Trajectory Evaluation*. PhD thesis, Carnegie Mellon University, August 1997.
37. P. Jain and G. Gopalakrishnan. Efficient symbolic simulation-based verification using the parametric form of Boolean expressions. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 13(11):1005–1015, August 1994.
38. R. B. Jones. *Applications of Symbolic Simulation to the Formal Verification of Microprocessors*. PhD thesis, Department of Electrical Engineering, Stanford University, August 1999.
39. R. B. Jones, J. W. O'Leary, C.-J. H. Seger, M. D. Aagaard, and T. F. Melham. Practical formal verification in microprocessor design. *IEEE Design & Test of Computers*, 18(4):16–25, July/August 2001.
40. J.J. Joyce and C.-J.H. Seger. Linking BDD-based symbolic evaluation to interactive theorem-proving. In *Design Automation Conference*, pages 469–474, 1993.
41. R. Kaivola. Formal verification of Pentium® 4 components with symbolic simulation and inductive invariants. In Kousha Etessami and Sriram K. Rajamani, editors, *Computer Aided Verification*, volume 3576 of *LNCS*, pages 170–184. Springer-Verlag, 2005.

42. R. Kaivola, R. Ghughal, N. Narasimhan, A. Telfer, J. Whittemore, S. Pandav, A. Slobodova, C. Taylor, V. Frolov, E. Reeber, and A. Naik. Replacing testing with formal verification in Intel Core i7 processor execution engine validation. In A. Bouajjani and O. Maler, editors, *Computer Aided Verification, CAV 2009: Proceedings*, Lecture Notes in Computer Science, pages 414–429. Springer-Verlag, 2009.
43. R. Kaivola and K. R. Kohatsu. Proof engineering in the large: formal verification of Pentium 4 floating-point divider. *International Journal on Software Tools for Technology Transfer*, 4(3):323–334, 2003.
44. M. Kaufmann and J.S. Moore. An industrial strength theorem prover for a logic based on common lisp. *IEEE Transactions on Software Engineering*, 23(4):203–213, 1997.
45. Z. Khasidashvili, G. Gavrielov, and T. Melham. Assume-guarantee validation for STE properties within an SVA environment. In Armin Biere and Carl Pixley, editors, *Proceedings of 9th International Conference: 2009 Formal Methods in Computer-Aided Design: FMCAD 2009*, pages 108–115. IEEE, 2009.
46. N. Krishnamurthy, A. K. Martin, M. S. Abadir, and J. A. Abraham. Validating PowerPC custom memories. *IEEE Design and Test of Computers*, 17(4):61–76, October/December 2000.
47. T. Melham. *Higher Order Logic and Hardware Verification*. Cambridge University Press, 1993.
48. T. F. Melham and R. B. Jones. Abstraction by symbolic indexing transformations. In Mark D. Aagaard and John W. O'Leary, editors, *Formal Methods in Computer-Aided Design: 4th International Conference, FMCAD 2002: Portland, OR, USA, November 6–8, 2002: Proceedings*, volume 2517 of *Lecture Notes in Computer Science*, pages 1–18. Springer-Verlag, 2002.
49. K. L. Nelson, A. Jain, and R. E. Bryant. Formal verification of a superscalar execution unit. In *Proceedings of the 34th annual Design Automation Conference*, pages 161–166. ACM, 1997.
50. K. Ng, A. J. Hu, and Yang J. Generating monitor circuits for simulation-friendly GSTE assertion graphs. In E. Grochowski and T. Dillinger, editors, *Proceedings of the IEEE International Conference on Computer Design: VLSI in Computers and Processors (ICCD 2004)*, pages 409–416. IEE Computer Society, 2004.
51. M. D. Nguyen, M. Thalmaier, M. Wedler, J. Bormann, D. Stoffel, , and W. Kunz. Unbounded protocol compliance verification using interval property checking with invariants. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 27(11):2068–2082, November 2008.
52. J. O'Leary, R. Kaivola, and T. Melham. Relational STE and theorem proving for formal verification of industrial circuit designs. In B. J. and S. Ray, editors, *FMCAD 2013: Formal Methods in Computer-Aided Design: Portland, Oregon, USA, 20–23 October 2013*, pages 97–104. IEEE, 2013.
53. M. Pandey, R. Raimi, R. E. Bryant, and M. S. Abadir. Formal verification of content addressable memories using symbolic trajectory evaluation. In *Design Automation Conference*, pages 167–172. ACM Press, June 1997.
54. J.-W. Roorda. *Semantics, Decision Procedures, and Abstraction Refinement for Symbolic Trajectory Evaluation*. PhD thesis, Chalmers University of Technology and Göteborg Universty, 2006.
55. J.-W. Roorda and K. Claessen. Explaining symbolic trajectory evaluation by giving it a faithful semantics. In D. Grigoriev, J. Harrison, and E. A. Hirsch, editors, *Computer Science - Theory and Applications, First International Computer Science Symposium in Russia, CSR 2006*, volume 3967 of *Lecture Notes in Computer Science*, pages 555–566. Springer-Verlag, 2006.
56. J.-W. Roorda and K. Claessen. SAT-based assistance in abstraction refinement for symbolic trajectory evaluation. In *Proceedings of the 18th international conference on Computer Aided Verification*, CAV'06, pages 175–189. Springer-Verlag, 2006.
57. M. Ryan and M. Sadler. Valuation systems and consequent relations. In S. Abramsky, D. M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 1, chapter 1, pages 1–78. Oxford University Press, 1992.

58. M. Santarini. Synopsys extends formal reach with InnoLogic acquisition. *EE Times*, June 2003. www.eetimes.com/document.asp?doc_id=1216962.
59. T. Schubert. High level formal verification of next-generation microprocessors. In *DAC'03: Proceedings of the 40th conference on design automation*, pages 1–6. ACM Press, 2003.
60. R. Sebastiani, E. Singerman, S. Tonetta, and M. Y. Vardi. GSTE is partitioned model checking. *Formal Methods in System Design*, 31(2):177–196, October 2007.
61. R. Sebastiani, S. Tonetta, , and M Vardi. Property-driven partitioning for abstraction refinement. In O. Grumberg and M. Huth, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, volume 4424 of *Lecture Notes in Computer Science*, pages 389–404. Springer-Verlag, 2007.
62. C.-J. H. Seger. Voss — a formal hardware verification system: User's guide. Technical Report TR-93-45, University of Brtish Columbia Department of Computer Science, December 1993.
63. C.-J. H. Seger, January 2014. Personal communication.
64. C.-J. H. Seger and R. E. Bryant. Formal verification by symbolic evaluation of partially-ordered trajectories. *Formal Methods in System Design*, 6(2):147–189, March 1995.
65. C.-J. H. Seger, R. B. Jones, J. W. O'Leary, T. Melham, M. D. Aagaard, C. Barrett, and D. Syme. An industrially effective environment for formal hardware verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 24(9):1381–1405, September 2005.
66. C.-J.H. Seger and J.J. Joyce. A mathematically precise two-level formal hardware verification methodology. Report 92-34, Department of Computer Science, University of British Columbia, December 1992.
67. A. Slobodová, J. Davis, S. Swords, and W. Hunt. A flexible formal verification framework for industrial scale validation. In *9th IEEE/ACM International Conference on Formal Methods and Models for Codesign*, pages 89–97. IEEE, 2011.
68. E. Smith. A logic for GSTE. In J. Baumgartner and M. Sheeran, editors, *Formal Methods in Computer Aided Design: FMCAD 2007: November 11–14 2007, Austin, Texas, USA*, pages 119–126. IEEE Computer Society, 2007.
69. E. Smith. *Specifying Properties for Generalized Symbolic Trajectory Evaluation*. PhD thesis, University of Oxford, March 2008.
70. R. Tzoref and O. Grumberg. Automatic refinement and vacuity detection for symbolic trajectory evaluation. In T. Ball and R. B. Jones, editors, *Computer Aided Verification, 18th International Conference, CAV 2006, Seattle, WA, USA, August 17-20, 2006, Proceedings*, volume 4144 of *Lecture Notes in Computer Science*, pages 190–204. Springer-Verlag, 2006.
71. M. N. Velev and R. E. Bryant. Efficient modeling of memory arrays in symbolic ternary simulation. In B. Steffen, editor, *Tools and Algorithms for the Construction and Analysis of Systems (TACAS '98)*, volume 1384 of *Lecture Notes in Computer Science*, pages 136–150. Springer-Verlag, 1998.
72. L.-C. Wang, M. S. Abadir, and N. Krishnamurthy. Automatic generation of assertions for formal verification of PowerPC microprocessor arrays using symbolic trajectory evaluation. In *Design Automation Conference, 1998: Proceedings*, pages 534–537. IEEE Computer Society, 1998.
73. J. Yang, R. Ghughal, and A. Tiemeyer. Industrial scale formal verification using concurrent GSTE. In T.-A. Tang and Y. Huang, editors, *International Conference on ASIC (ASICON'05)*, pages 18–032. IEEE Computer Society, 2005.
74. J. Yang and A. Goel. GSTE through a case study. In L. Pileggi and A. Kuelmann, editors, *IEEE/ACM International Conference on Computer Aided Design*, pages 534–541. ACM Press, 2002.
75. J. Yang and C.-H. Seger. Generalized symbolic trajectory evaluation—abstraction in action. In Mark D. Aagaard and John W. O'Leary, editors, *Formal Methods in Computer-Aided Design: 4th International Conference, FMCAD 2002: Portland, OR, USA, November 6–8, 2002: Proceedings*, volume 2517 of *Lecture Notes in Computer Science*, pages 70–87. Springer-Verlag, 2002.

76. J. Yang and C.-J. Seger. Compositional specification and model checking in GSTE. In R. Alur and D. A. Peled, editors, *Computer Aided Verification*, volume 3114 of *Lecture Notes in Computer Science*, pages 216–228. Springer-Verlag, 2004.
77. J. Yang and C.-J. H. Seger. Generalized symbolic trajectory evaluation. Technical report, Intel Strategic CAD Labs, 2000.
78. J. Yang and C.-J. H. Seger. Introduction to generalized symbolic trajectory evaluation. *IEEE Transactions on VLSI Systems*, 11(3):345–353, June 2003.
79. J. X. Zhong. Circuit simulation using encoding of repetitive subcircuits, December 2001. US Patent Number 6,865,525.
80. Z. Zhu and C.-J. Seger. The completeness of a hardware inference system. In D. L. Dill, editor, *Computer Aided Verification, 6th International Conference, CAV'94: Proceedings*, volume 818 of *Lecture Notes in Computer Science*, pages 286–298. Springer-Verlag, 1994.

Index